

Optimizing Memory Layouts and Traversal Orders in HPC

Jiří Klepl

Department of Distributed and Dependable Systems

Motivation



Motivation



- Simplify and streamline algorithm design for scientific computing

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops
 - Optimizing memory accesses for best performance

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops
 - Optimizing memory accesses for best performance

Research Questions

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops
 - Optimizing memory accesses for best performance

Research Questions

1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops
 - Optimizing memory accesses for best performance

Research Questions

1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?
2. Can developers write abstract computations and choose optimizations of value encoding and evaluation in specific domains?

Motivation



- Simplify and streamline algorithm design for scientific computing
 - Regular data structures in nested loops
 - Optimizing memory accesses for best performance

Research Questions

1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?
2. Can developers write abstract computations and choose optimizations of value encoding and evaluation in specific domains?
3. How do design guidance and abstractions affect the performance and validity of LLM-generated code?



- **Memory access transformations**
 - **Decoupling layouts and traversals from the algorithm**
 - Parallelization of traversals
 - Distributed computing
- Decoupling value encoding and evaluation strategy
- Design guidance and abstractions' effect on LLM-generated code

Publications

Astute Approach to Handling Memory Layouts of Regular Data Structures. *ICA3PP*, 2022

Pure C++ Approach to Optimized Parallel Traversal of Regular Data Structures. *PPoPP*, 2024

Abstractions for C++ code optimizations in parallel high-performance applications. *ParCo*, 2024

Layout-Agnostic MPI Abstraction for Distributed Computing in Modern C++. *EuroMPI*, 2025

Why layouts matter



$m \times n$ matrix initialization

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Row-by-row traversal

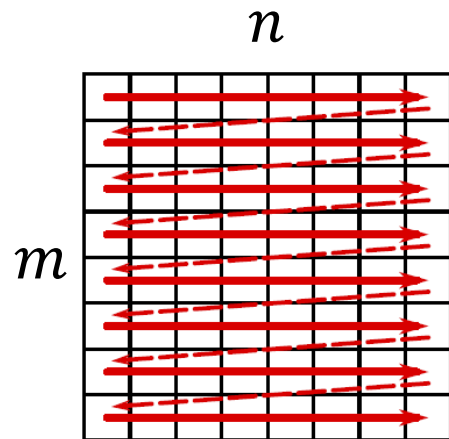
Why layouts matter



$m \times n$ matrix initialization

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Row-by-row traversal



Row-major

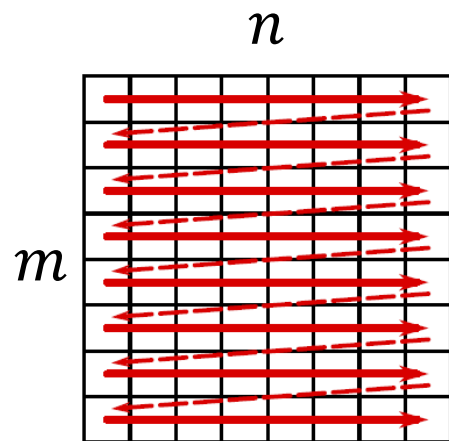
Why layouts matter



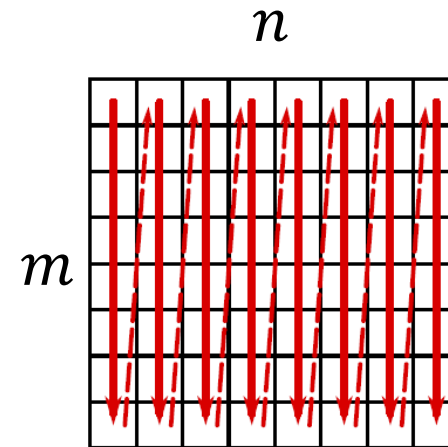
$m \times n$ matrix initialization

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Row-by-row traversal



Row-major



Column-major



Layout-agnostic design



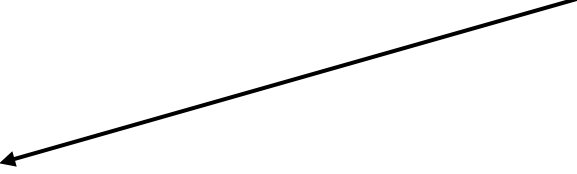
```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Layout-agnostic design



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Index space

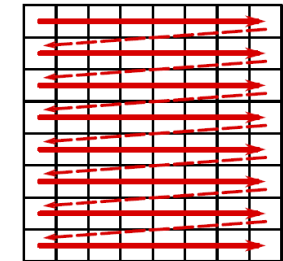
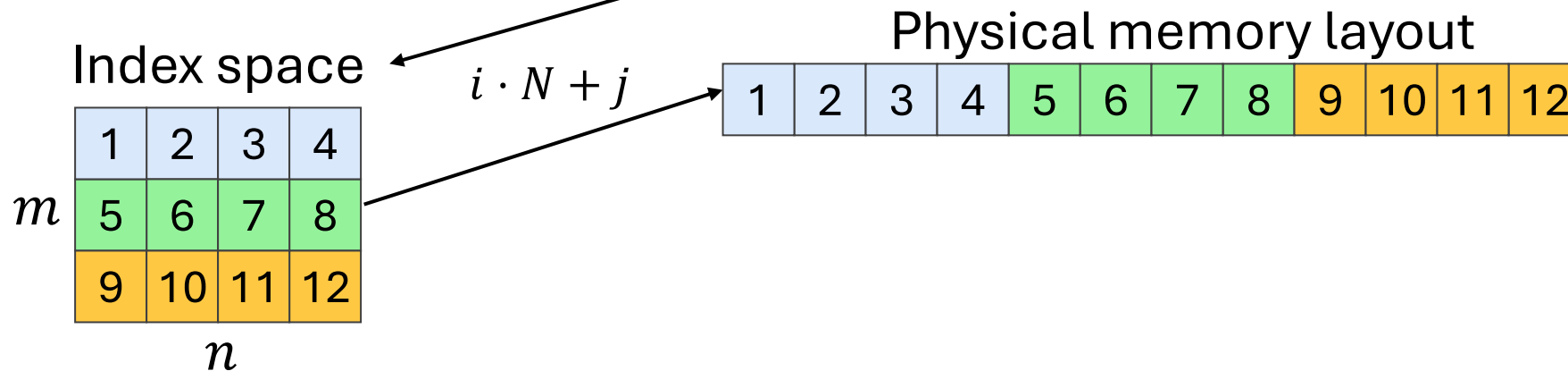


	1	2	3	4
m	5	6	7	8
	9	10	11	12
	n			

Layout-agnostic design



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

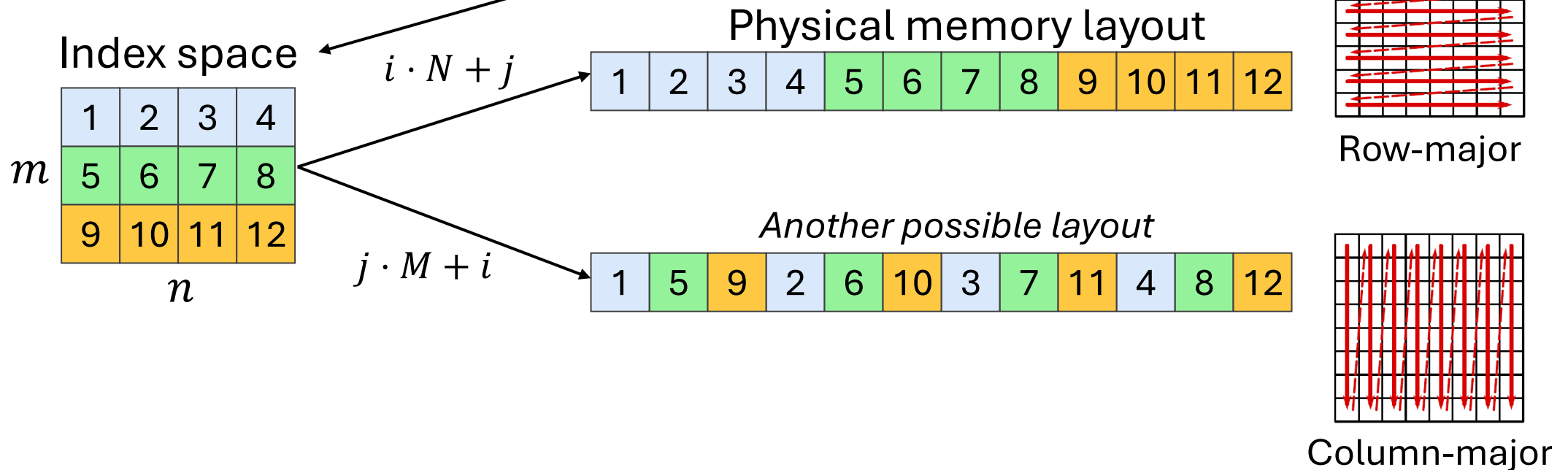


Row-major

Layout-agnostic design



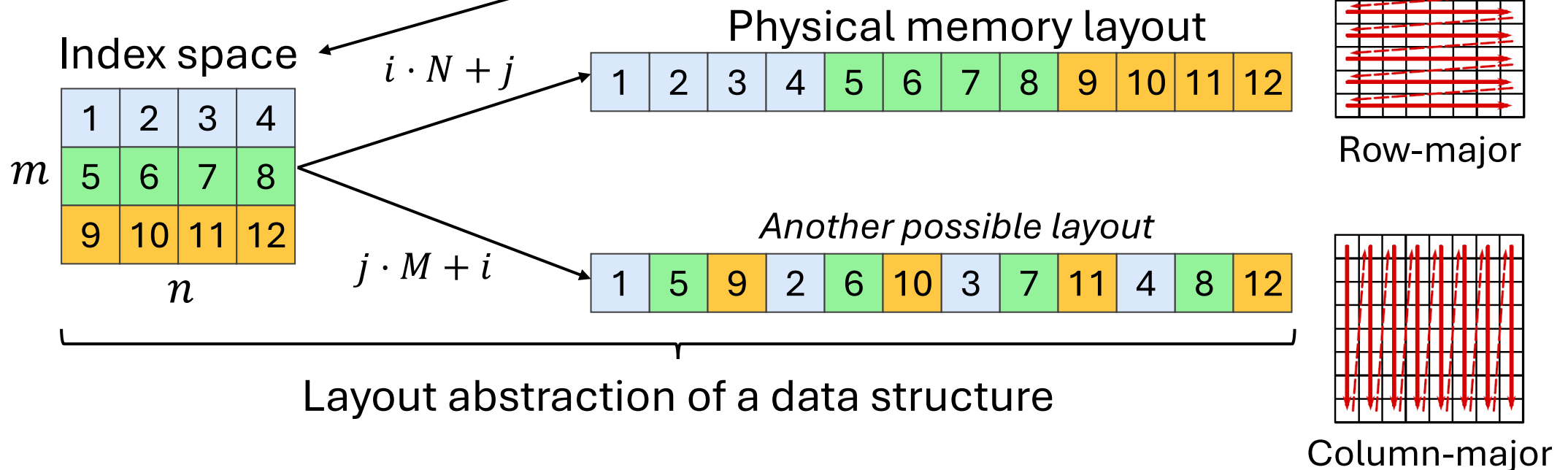
```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```



Layout-agnostic design



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```



Layout abstractions (Noarr)



Composition of building blocks in native C++

Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names



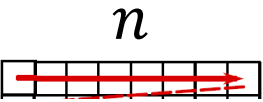
```
layout = scalar<float>()
```

Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names

 `layout = scalar<float>() ^`
`vector<'n'>(N)`

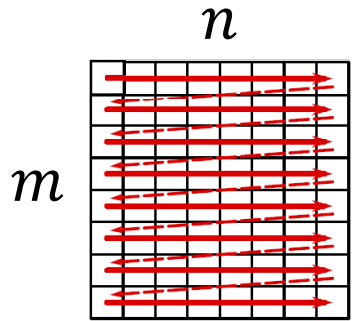
The diagram shows a 1D array layout with dimension n . It consists of a horizontal row of 10 small squares. A red arrow points from the first square to the last square, indicating the extent of the dimension. The variable `layout` is assigned the composition of a `scalar<float>()` and a `vector<'n'>(N)`.

Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names



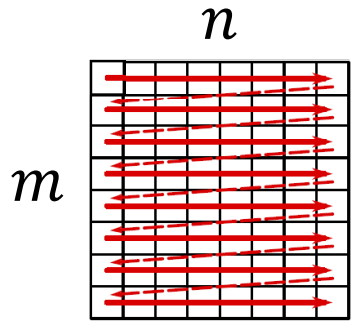
```
layout = scalar<float>() ^  
          vector<'n'>(N) ^  
          vector<'m'>(M);
```

Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names



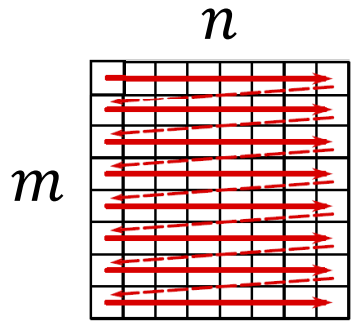
```
layout = scalar<float>() ^  
          vector<'n'>(N) ^  
          vector<'m'>(M); ^  
                                access  
                                matrix[idx<'m','n'>(4,2)]
```

Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names



```
layout = scalar<float>() ^  
         vector<'n'>(N) ^  
         vector<'m'>(M); ^  
                           access  
                           matrix[idx<'m','n'>(4,2)]
```

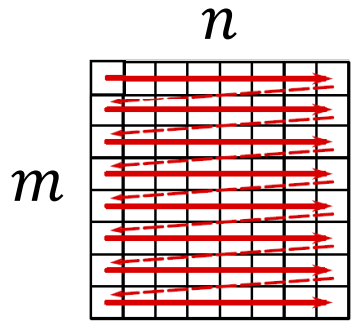
```
auto matrix = bag(layout);
```


Layout abstractions (Noarr)



Composition of building blocks in native C++

Parameters specifying types and dimension names



```
layout = scalar<float>() ^  
         vector<'n'>(N) ^  
         vector<'m'>(M); ^  
                        access  
                        matrix[idx<'m','n'>(4,2)]
```

GPU mem.

Mapped file

...

```
auto matrix = bag(layout);
```

```
auto matrix_view = bag(layout, myBuffer);
```

Traversal abstractions



Matrix initialization (not traversal-agnostic)

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

Traversal abstractions



Matrix initialization (not traversal-agnostic)

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

`idx < 'm', 'n' > (...)`

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Traversal-agnostic Matrix initialization

Traversal abstractions



Matrix initialization (not traversal-agnostic)

```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        matrix[i][j] = init(i, j);
```

idx < 'm', 'n' > (...)

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Traversal-agnostic Matrix initialization

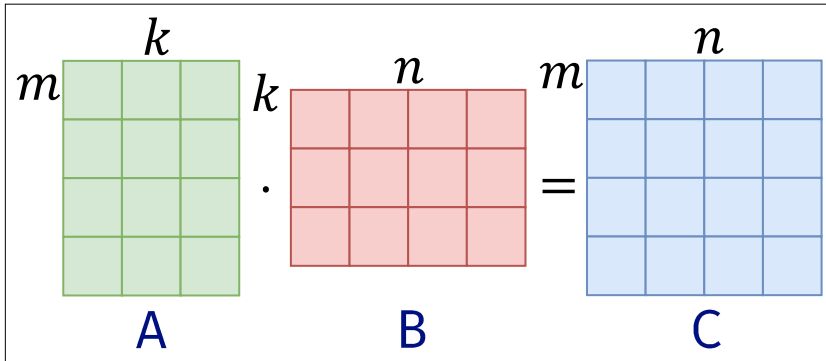
```
traverser(matrix, other_matrix).for_each([&](auto index) {  
    other_matrix[index] = matrix[index];  
});
```

Copy to another matrix with the same index space

Structures with different index spaces



Matrix multiplication

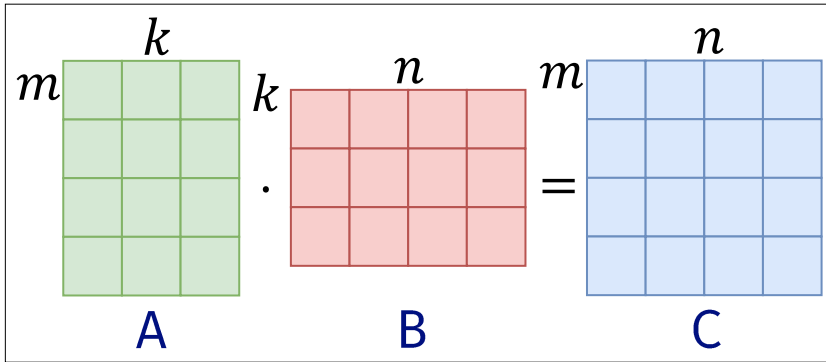


```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

Structures with different index spaces



Matrix multiplication



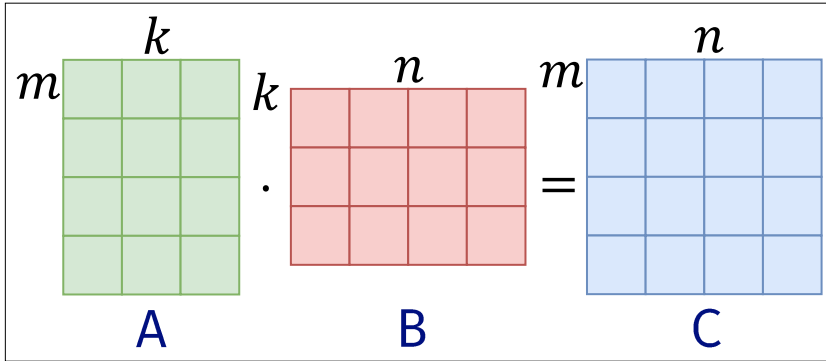
```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

Structures with different index spaces



Matrix multiplication



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

A: $m \times k$

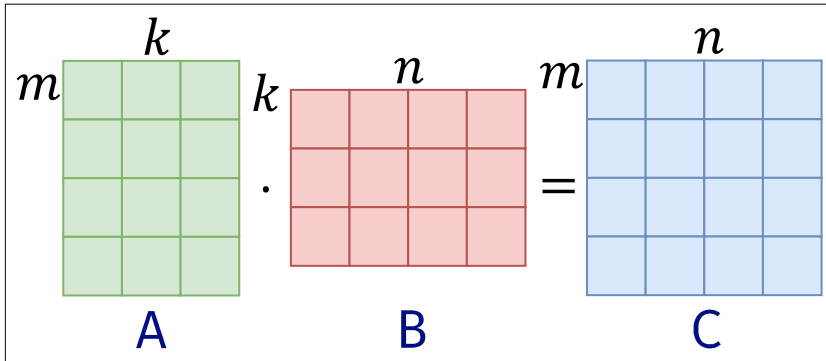
B: $k \times n$

C: $m \times n$

Structures with different index spaces



Matrix multiplication



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

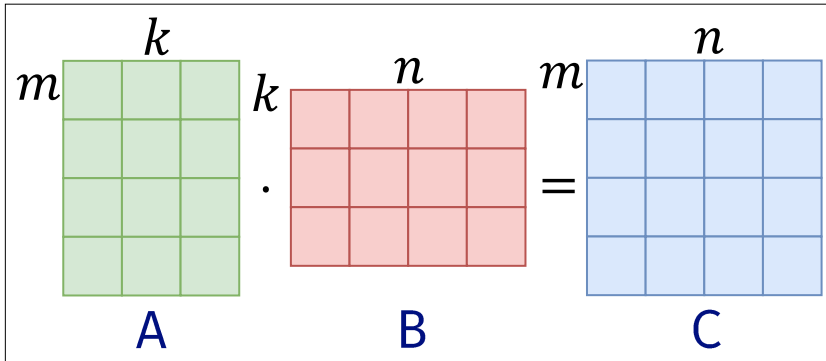
```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

A: $m \times k$
B: $k \times n$
C: $m \times n$

Structures with different index spaces

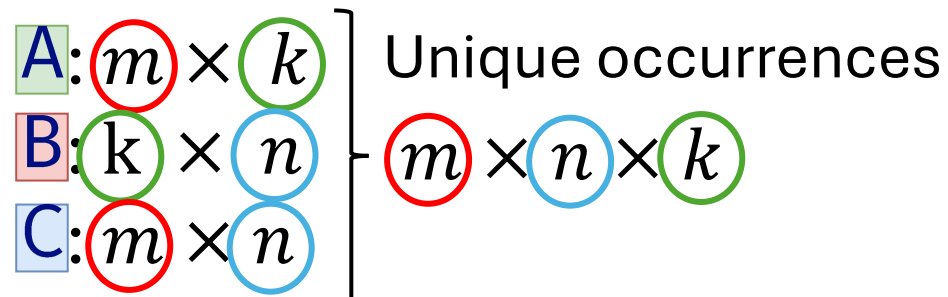


Matrix multiplication



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

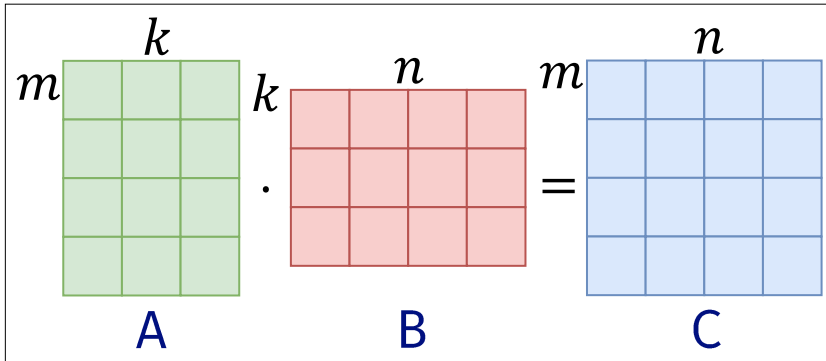
```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```



Structures with different index spaces

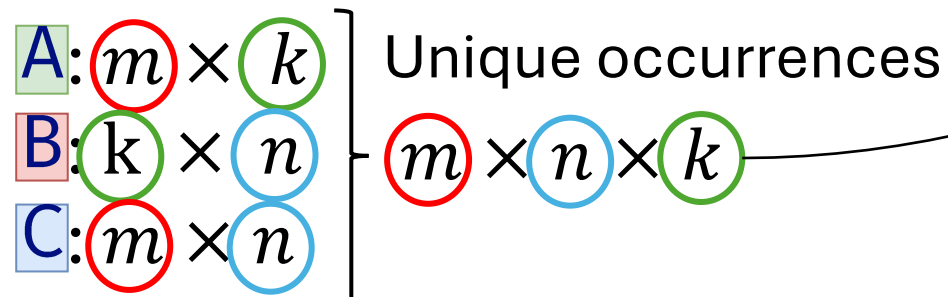


Matrix multiplication



```
for (int i = 0; i < M; i++)  
    for (int j = 0; j < N; j++)  
        for (int k = 0; k < K; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

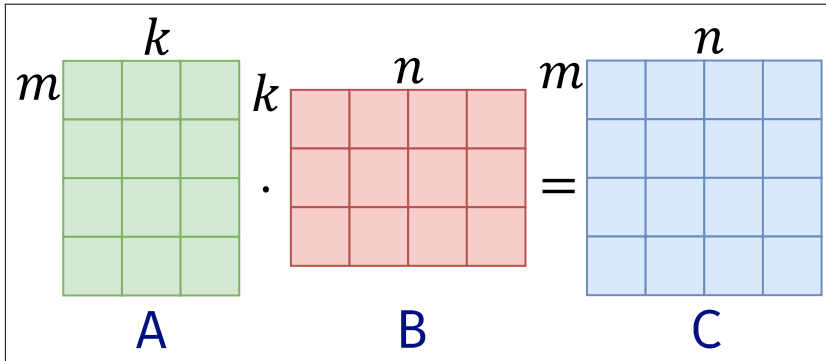
```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```



Structures with different index spaces



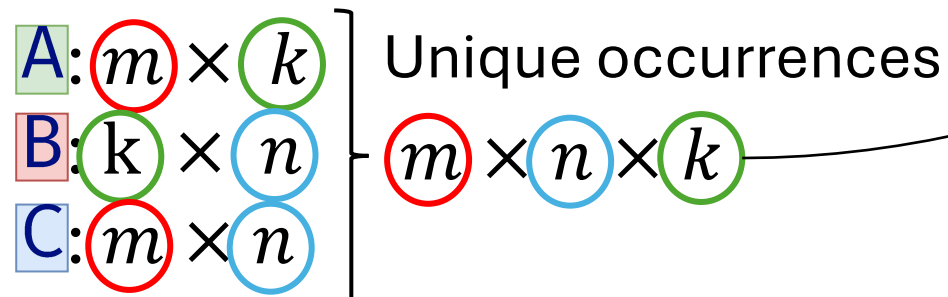
Matrix multiplication



```
for (int i = 0; i < M; i++)  
  for (int j = 0; j < N; j++)  
    for (int k = 0; k < K; k++)  
      C[i][j] += A[i][k] * B[k][j];
```

```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

Correct indices passed automatically

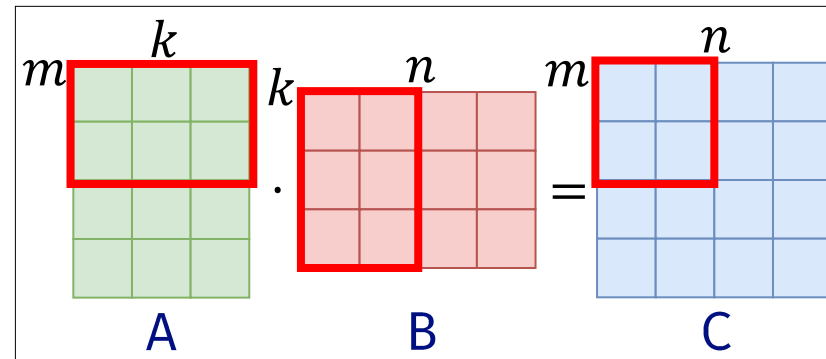


Same computation, different traversal



```
traverser(A, B, C).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

Common optimization: compute by tiles



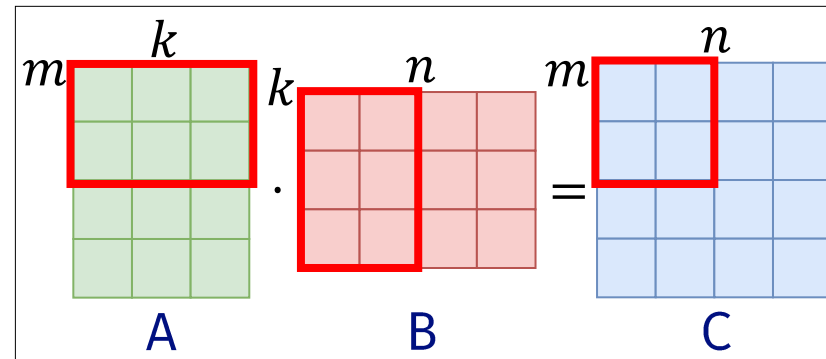
Same computation, different traversal



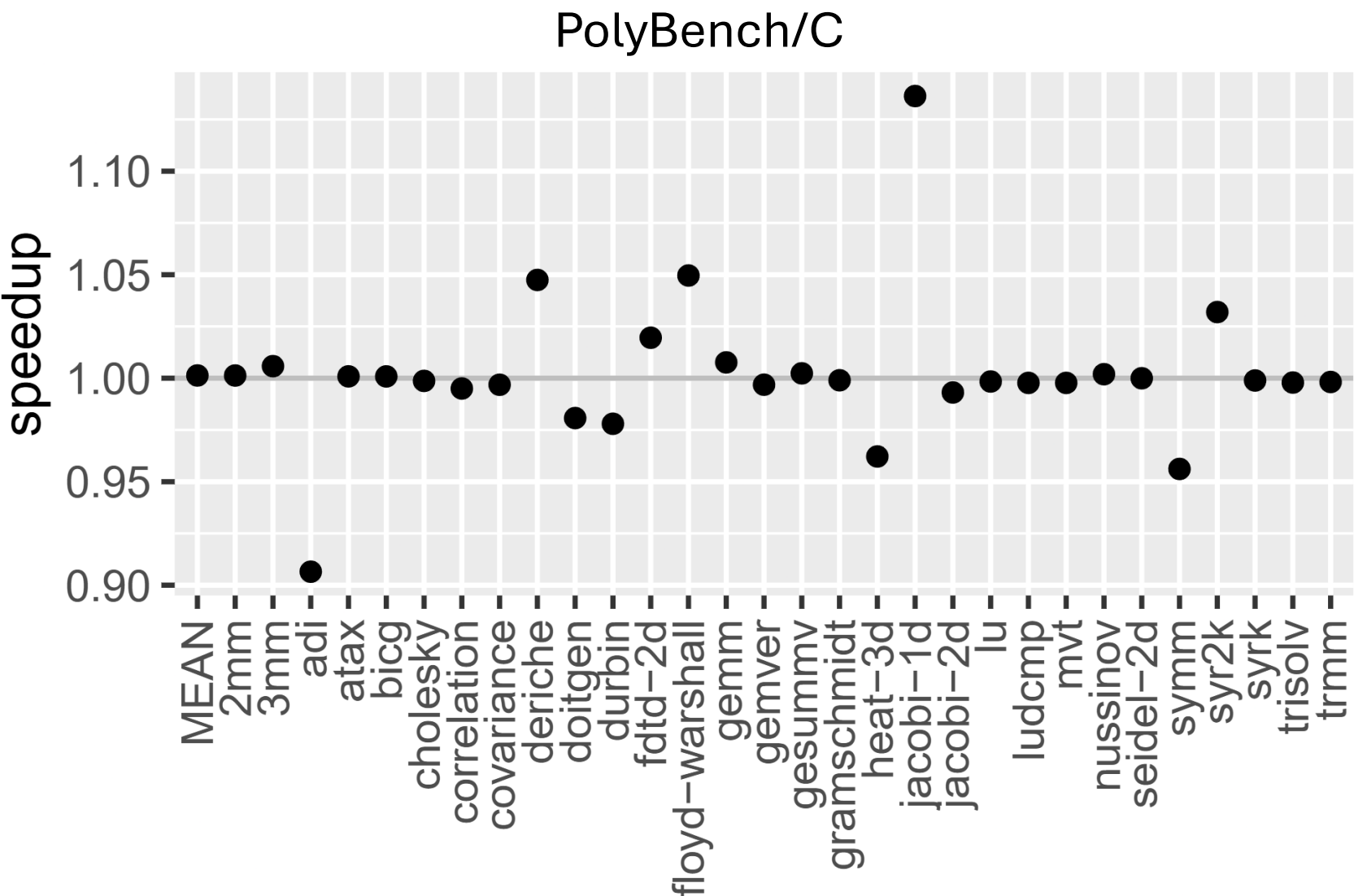
```
auto by_tiles = into_blocks <'m', 'M'> (16) ^  
                into_blocks <'n', 'N'> (16) ^  
                hoist <'M', 'N'> ();
```

```
(traverser(C, B, A) ^ by_tiles).for_each([&](auto index) {  
    C[index] += A[index] * B[index];  
});
```

Common optimization: compute by tiles



No measurable runtime overhead





- **Memory access transformations**

- Decoupling layouts and traversals from the algorithm
- **Parallelization of traversals**
- Distributed computing
- Decoupling value encoding and evaluation strategy
- Design guidance and abstractions' effect on LLM-generated code

Publications

Astute Approach to Handling Memory Layouts of Regular Data Structures. *ICA3PP*, 2022

Pure C++ Approach to Optimized Parallel Traversal of Regular Data Structures. *PPoPP*, 2024

Abstractions for C++ code optimizations in parallel high-performance applications. *ParCo*, 2024

Layout-Agnostic MPI Abstraction for Distributed Computing in Modern C++. *EuroMPI*, 2025

Traversal parallelization



- CPU

- OpenMP

- `omp_for_each(traverser, ...)`

- oneTBB

- `tbb_for_each(traverser, ...)`

Traversal parallelization



- CPU

- OpenMP

- `omp_for_each(traverser, ...)`

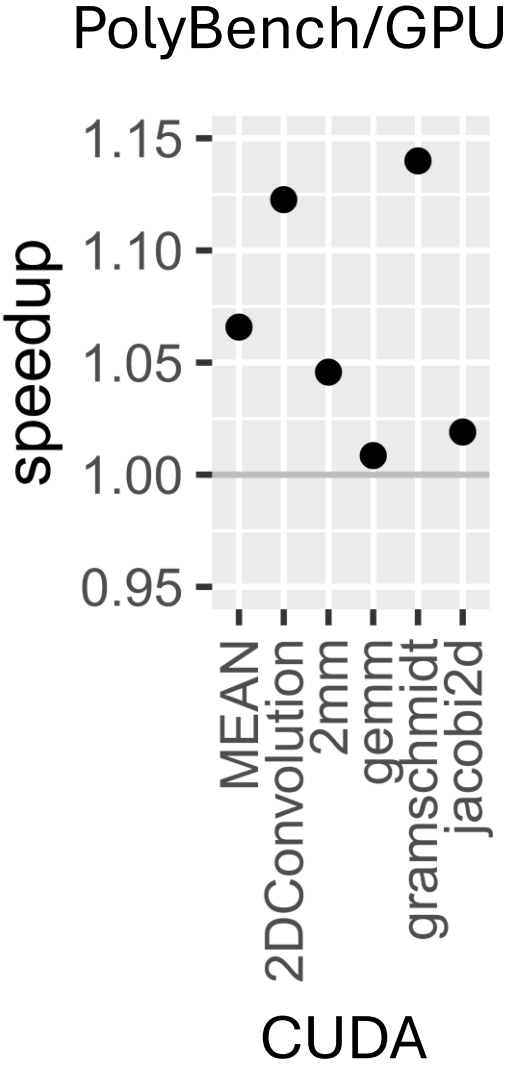
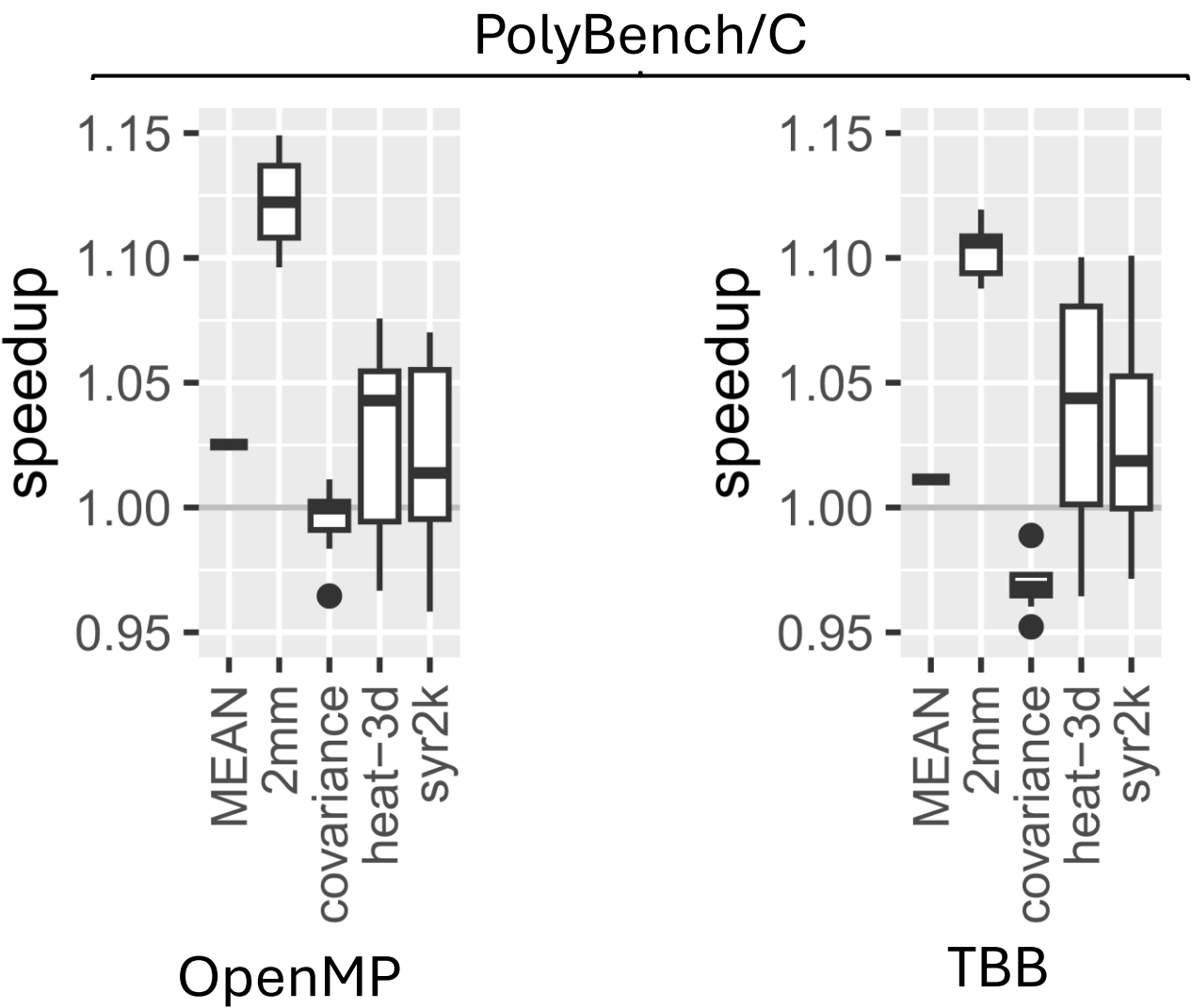
- oneTBB

- `tbb_for_each(traverser, ...)`

- GPU

- `cuda_traverser = cuda_threads<'t', 'b'>(traverser)`
`cuda_traverser.simple_run(...)`

No overhead in parallelization





- **Memory access transformations**

- Decoupling layouts and traversals from the algorithm
- Parallelization of traversals
- **Distributed computing**
- Decoupling value encoding and evaluation strategy
- Design guidance and abstractions' effect on LLM-generated code

Publications

Astute Approach to Handling Memory Layouts of Regular Data Structures. *ICA3PP*, 2022

Pure C++ Approach to Optimized Parallel Traversal of Regular Data Structures. *PPoPP*, 2024

Abstractions for C++ code optimizations in parallel high-performance applications. *ParCo*, 2024

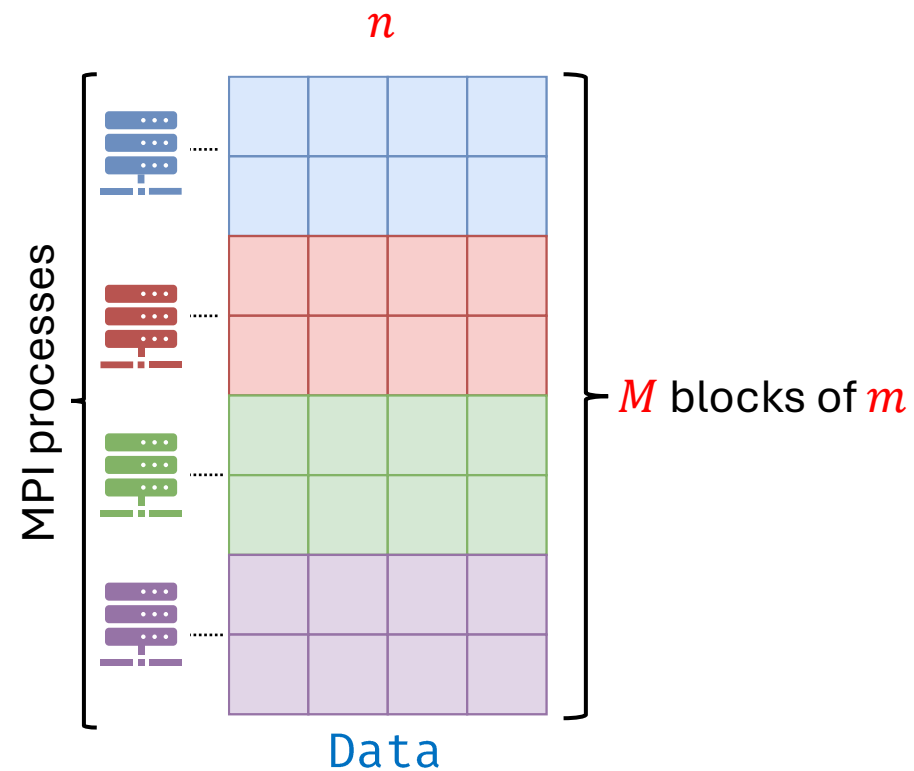
Layout-Agnostic MPI Abstraction for Distributed Computing in Modern C++. *EuroMPI*, 2025

Distributed initialization using MPI



We want to distribute:

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```



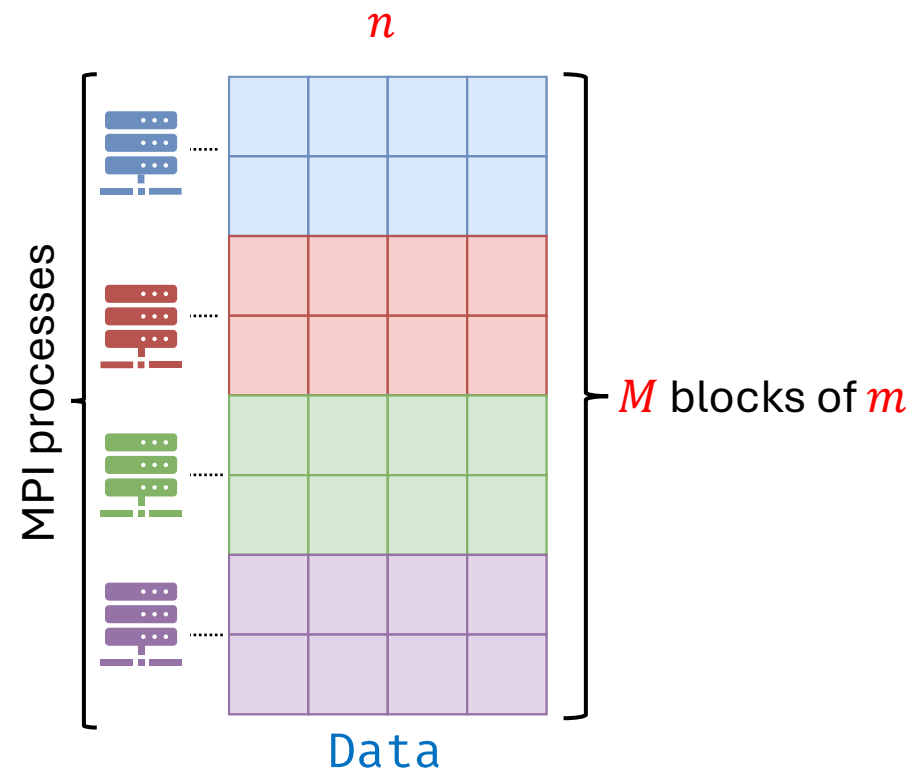
Distributed initialization using MPI



We want to distribute:

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Binding traversal to an MPI communicator



Distributed initialization using MPI

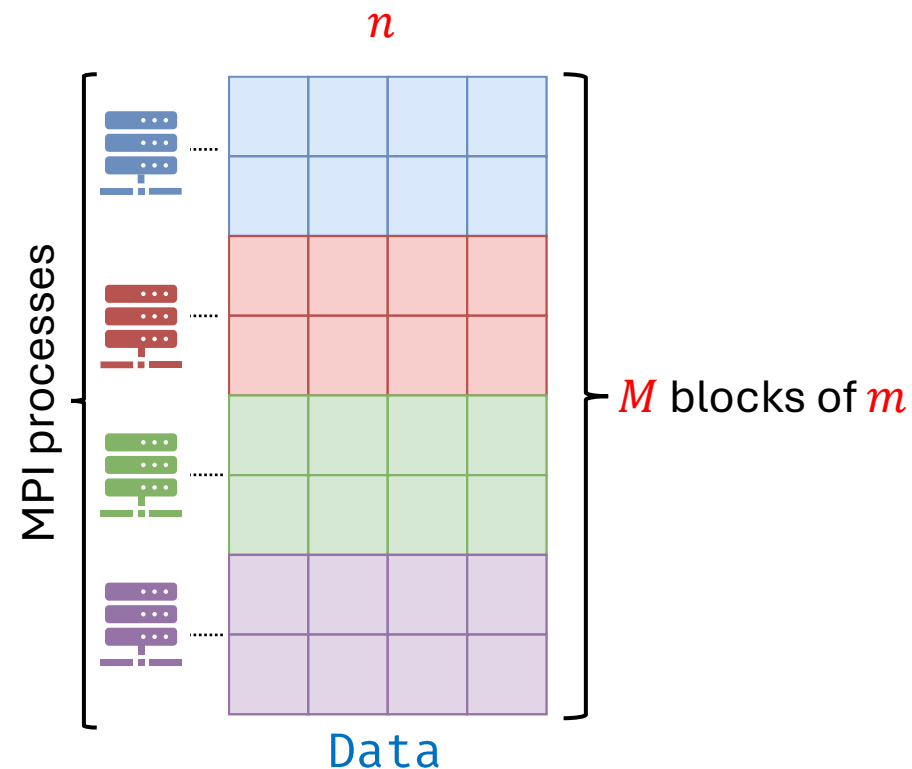


We want to distribute:

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Binding traversal to an MPI communicator

```
auto MPItrav =  
    mpi_traverser<'M'>(traverser, MPI_COMM_WORLD);
```



Distributed initialization using MPI

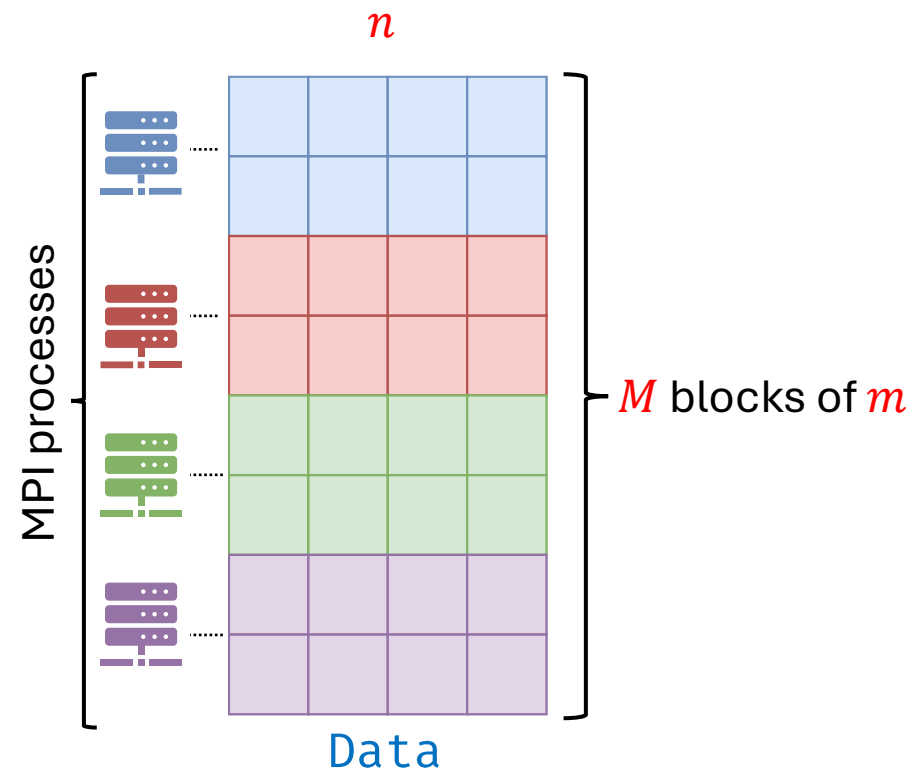


We want to distribute:

```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Binding traversal to an MPI communicator

```
auto MPItrav =  
    mpi_traverser<'M'>(traverser, MPI_COMM_WORLD);  
  
auto Tile = ...  
MPItrav.for_each([&](auto index) {  
    Tile[index] = init(index);  
});
```



Distributed initialization using MPI

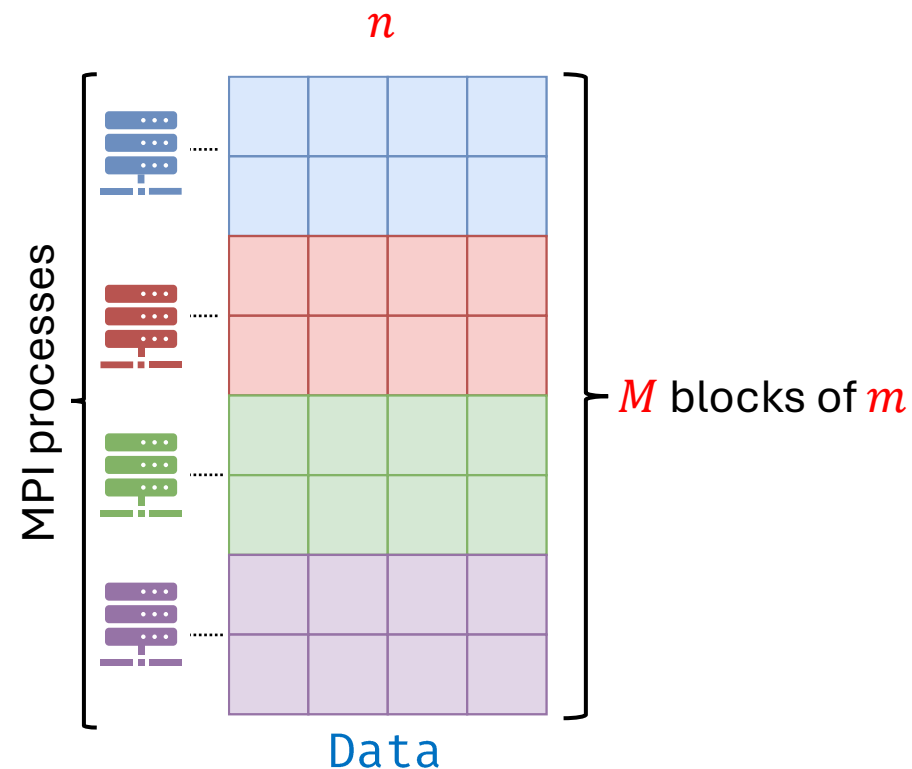


We want to distribute:

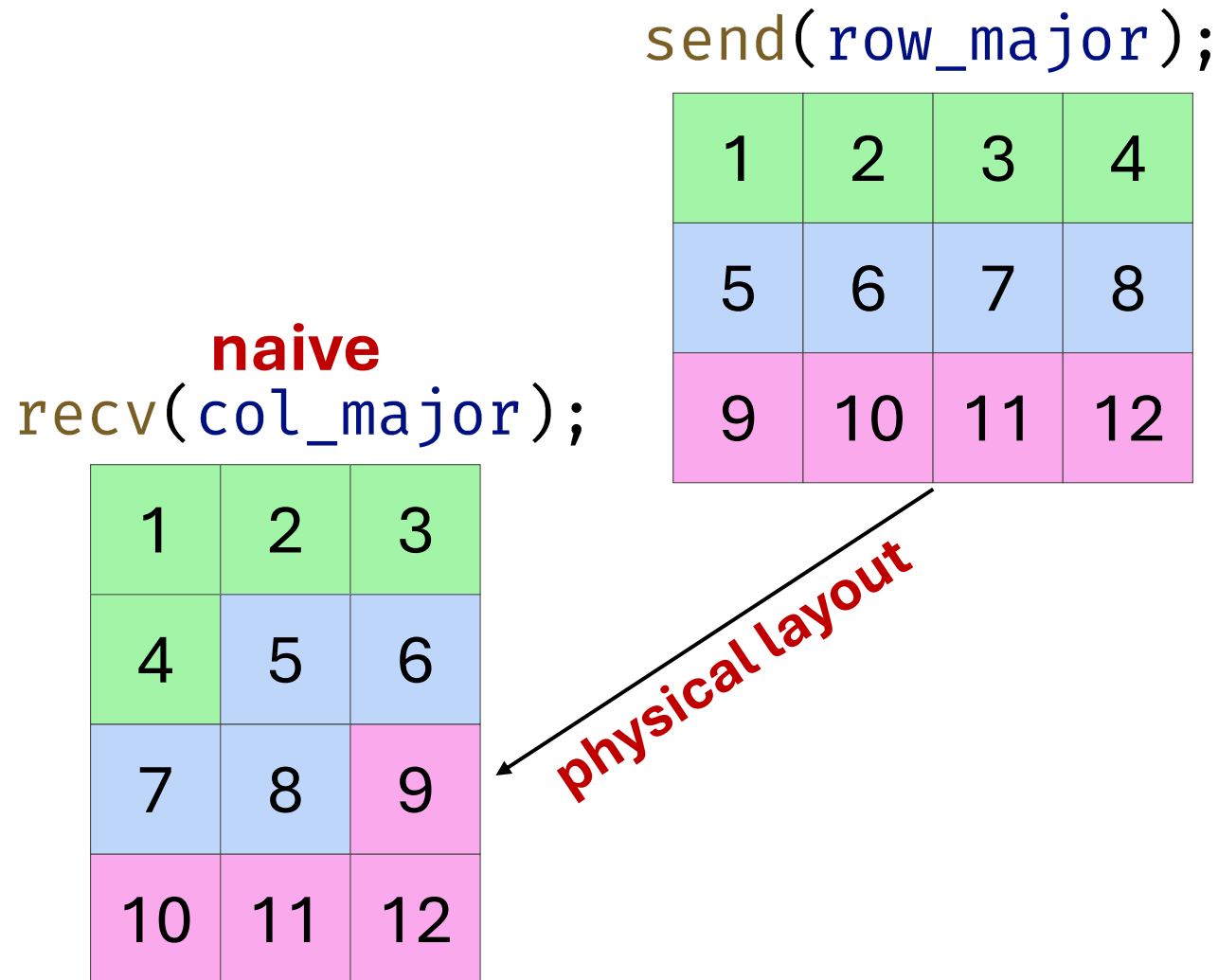
```
traverser(matrix).for_each([&](auto index) {  
    matrix[index] = init(index);  
});
```

Binding traversal to an MPI communicator

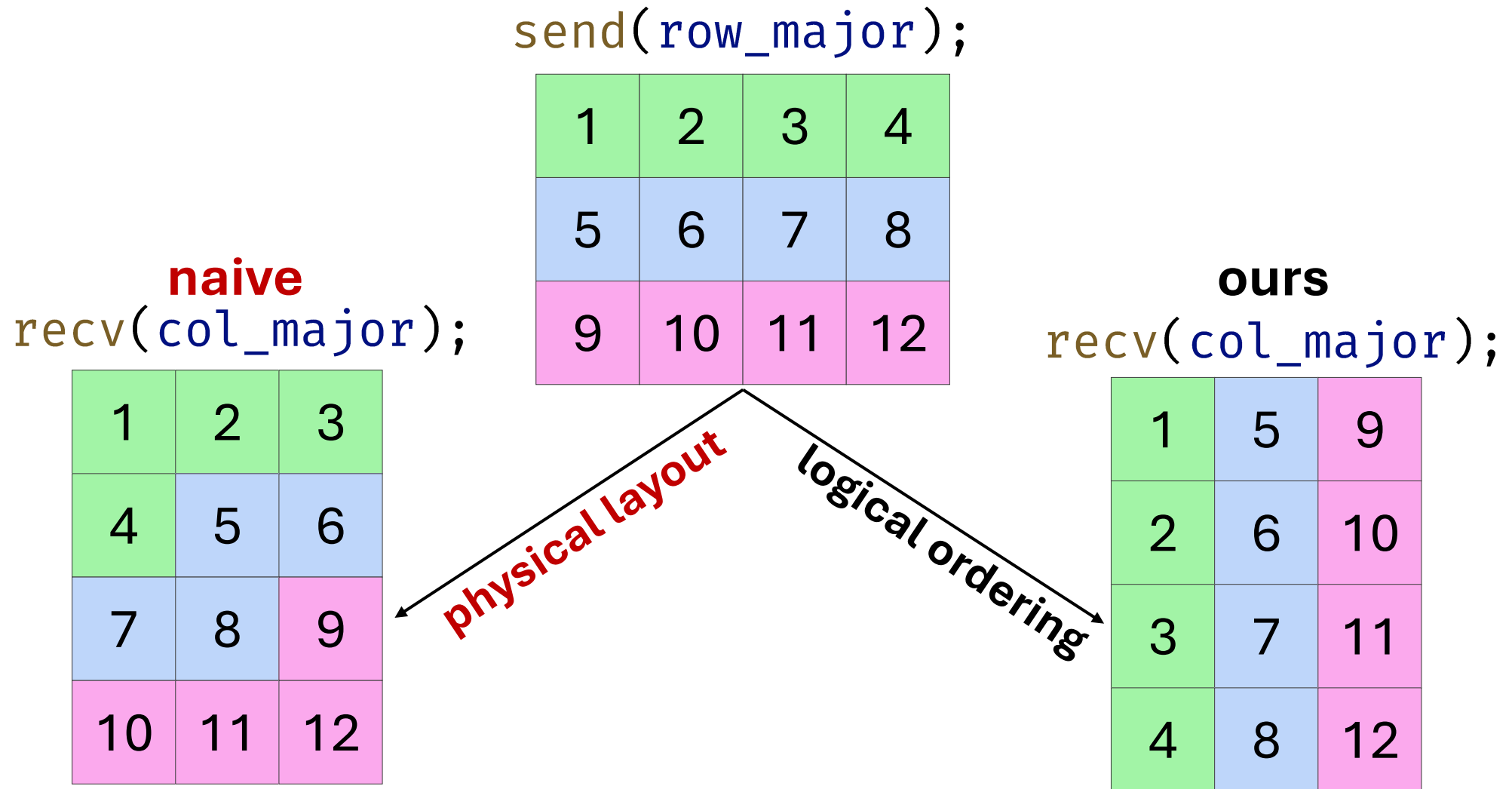
```
auto MPItrav =  
    mpi_traverser<'M'>(traverser, MPI_COMM_WORLD);  
  
auto Tile = ...  
MPItrav.for_each([&](auto index) {  
    Tile[index] = init(index);  
});  
  
gather(Tile, Data, root, MPItrav);
```



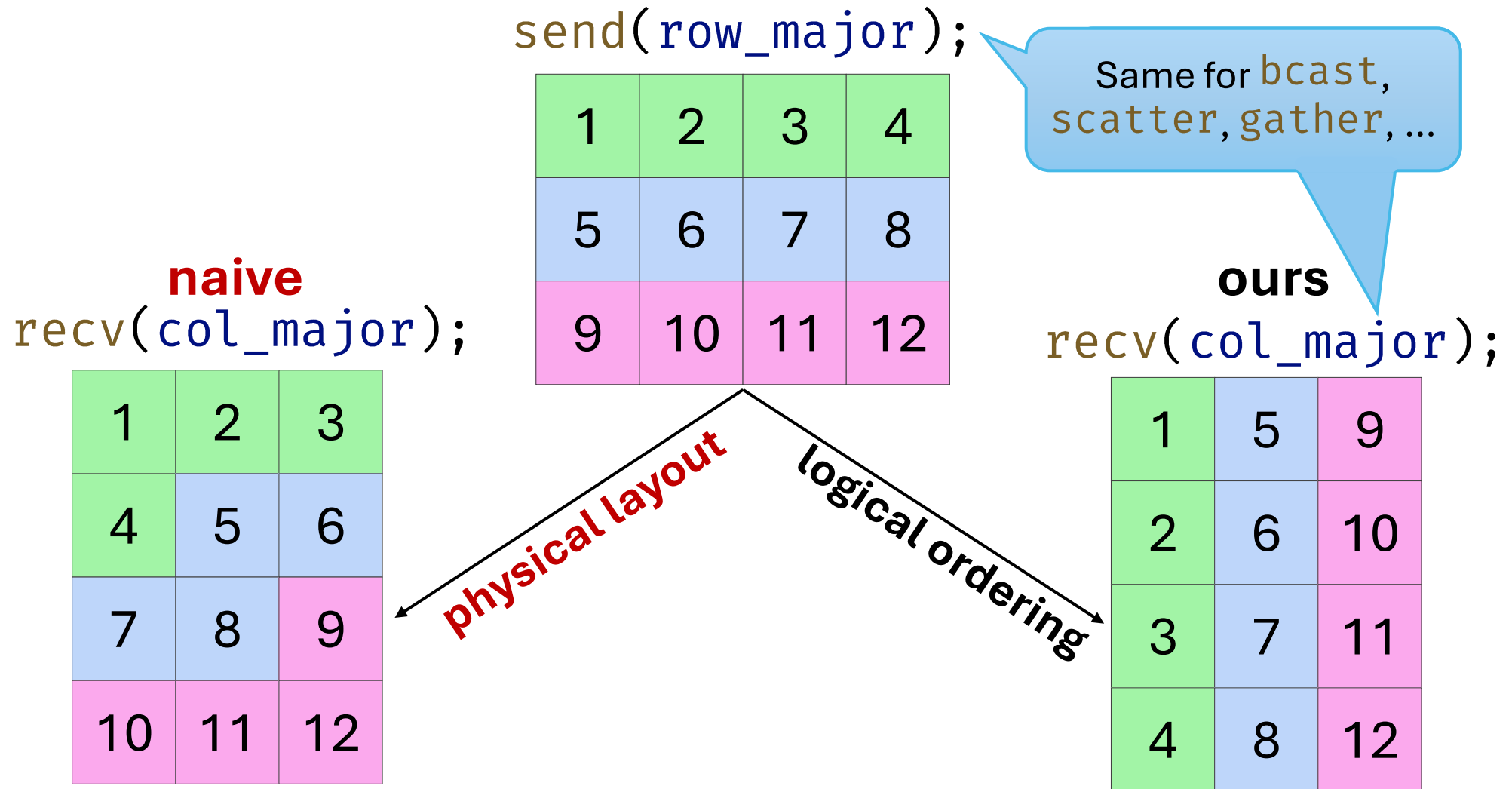
Automatic transformation in communication



Automatic transformation in communication



Automatic transformation in communication

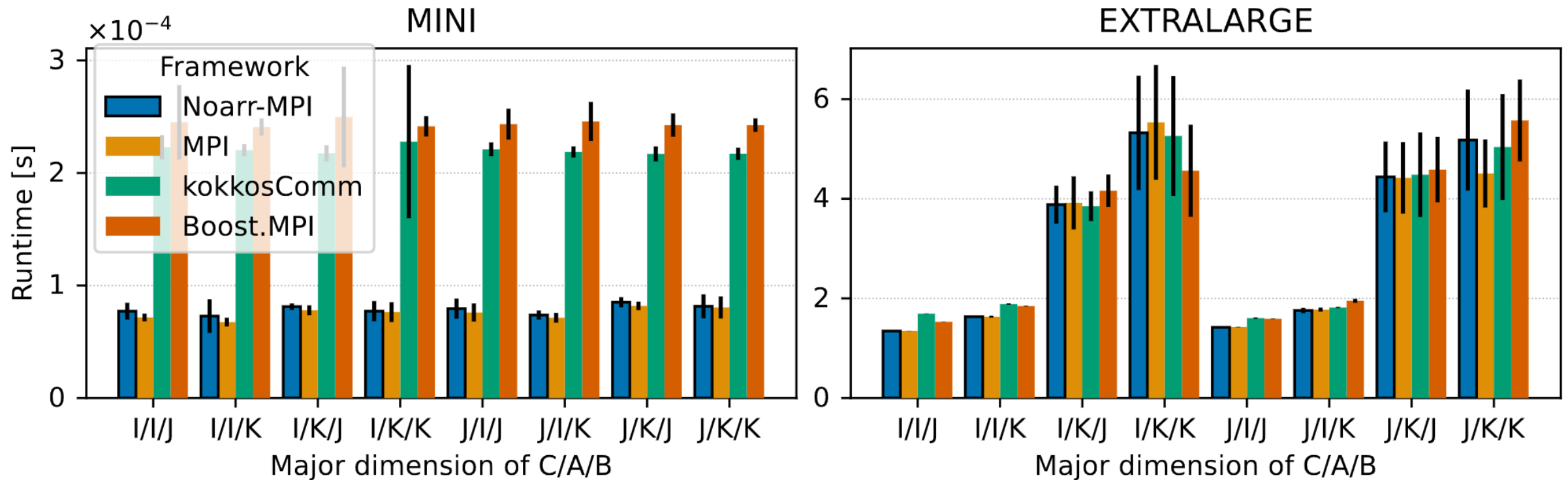


Support for layout-agnostic data distribution



	Noarr-MPI	native MPI	Boost.MPI	MPP	MPL	Kokkos	KaMPIng
Auto-transforms	✓	★	✗	✗	✗	✗	✗
Non-contiguous	✓	✓	✓	✓	✓	✓	✗
Mdspan-like	✓	✗	✗	✗	✗	✓	✗
Seamless	✓	✓	✗	✓	✓	✓	✓
Type-safety	✓	✗	✓	✓	✓	✓	✓
Scatter/gather	✓	✓	✗	✗	✓	✗	✗

Noarr-MPI does not introduce overhead





- Memory access transformations
- **Decoupling value encoding and evaluation strategy**
- Design guidance and abstractions' effect on LLM-generated code

Publications

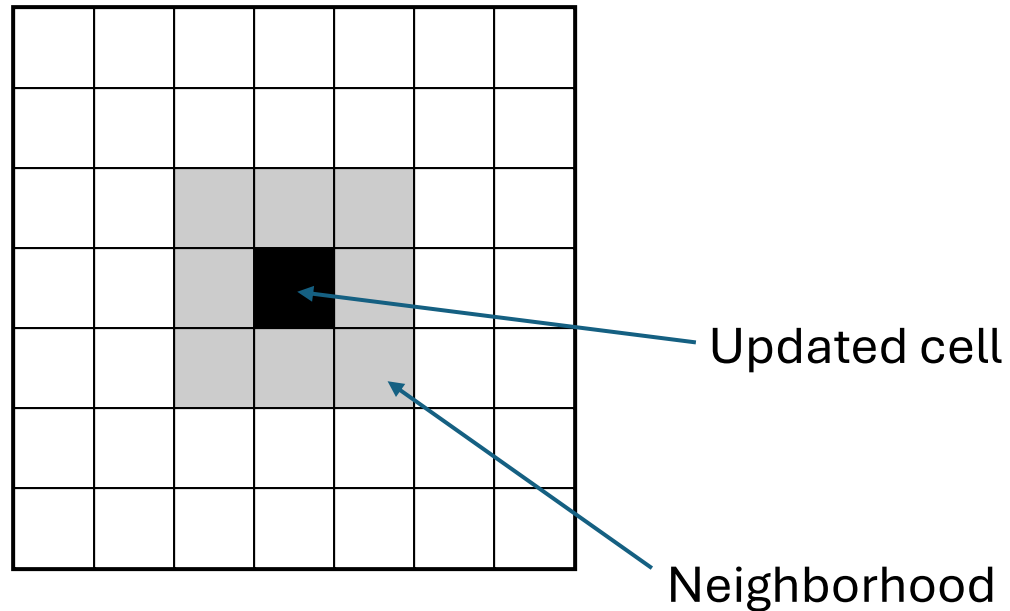
Cellato: a DSL for Cellular Automata based on Template Metaprogramming. JOT, 2026

Improving Cellular Automata Performance. ParCo, 2026 (submitted)

Stencil computation abstractions



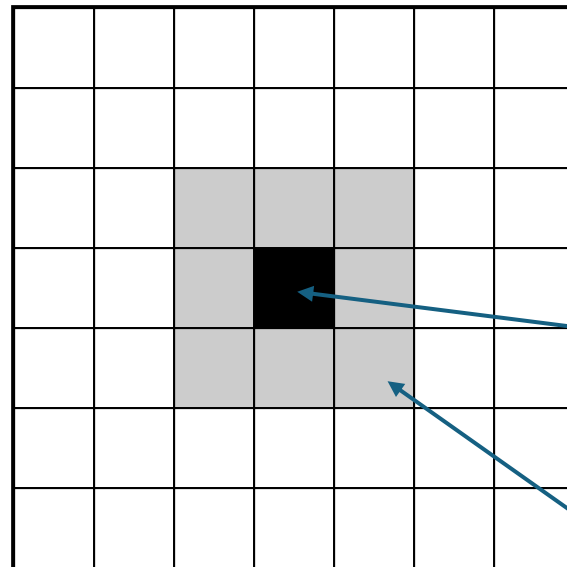
Example of an 8-point stencil



Stencil computation abstractions

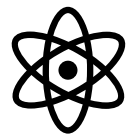


Example of an 8-point stencil



Updated cell

Neighborhood



Physics simulation

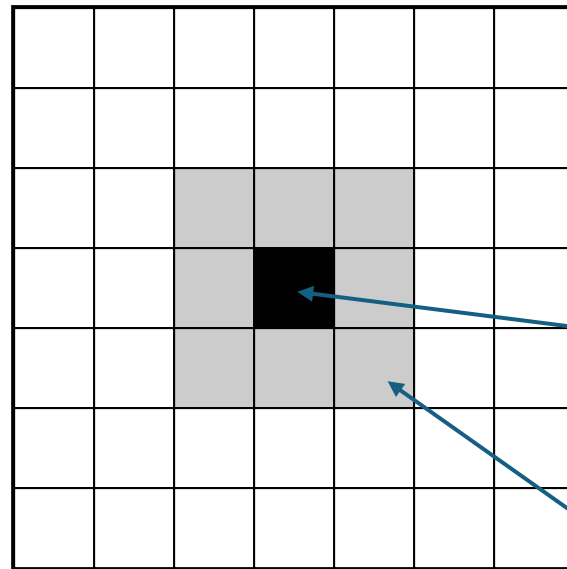


Image processing

Stencil computation abstractions



Example of an 8-point stencil



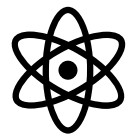
Updated cell

Neighborhood

Can we abstract more?



Colleague Brabec

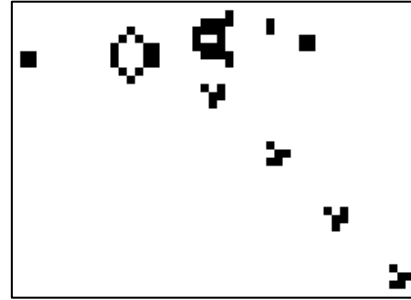


Physics simulation

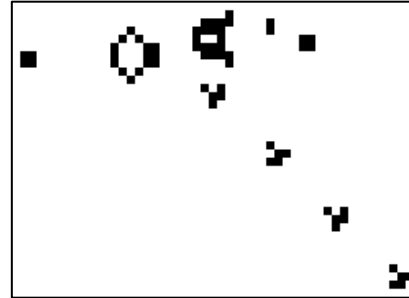


Image processing

Discrete stencils – Cellular automata

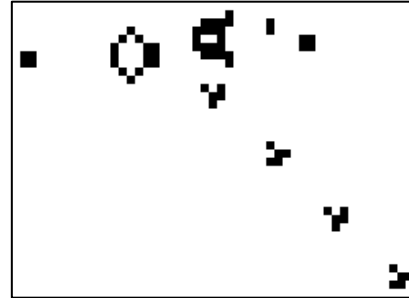


Discrete stencils – Cellular automata



Discrete state space
Neighbor-count rules

Discrete stencils – Cellular automata

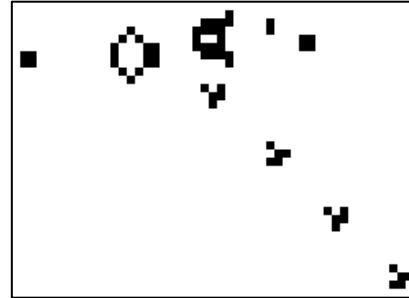


Discrete state space
Neighbor-count rules

Update rule for Game of Life

```
if (current == ALIVE) {  
    if (neighbors == 3 || neighbors == 2)  
        next = ALIVE;  
    else  
        next = DEAD;  
} else if (neighbors == 3)  
    next = ALIVE;  
else  
    next = DEAD;
```

Discrete stencils – Cellular automata



Discrete state space
Neighbor-count rules

Update rule for Game of Life

```
if (current == ALIVE) {  
  if (neighbors == 3 || neighbors == 2)  
    next = ALIVE;  
  else  
    next = DEAD;  
} else if (neighbors == 3)  
  next = ALIVE;  
else  
  next = DEAD;
```

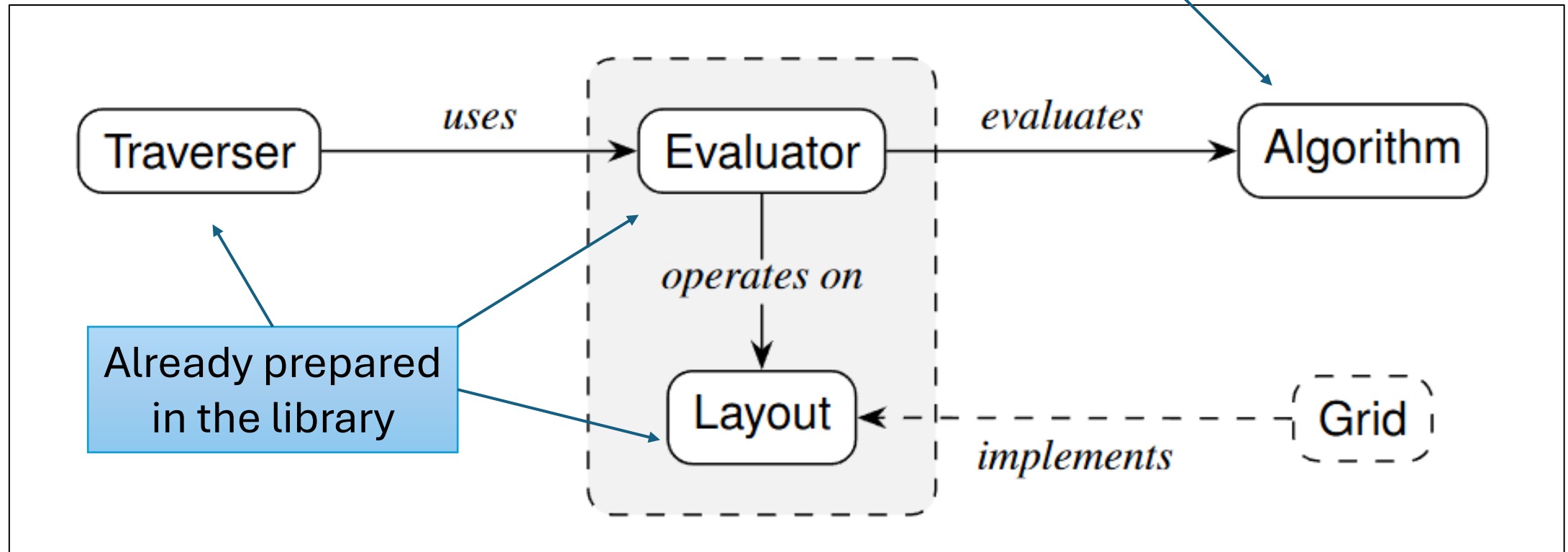
Type-level expression (Cellato DSL)

```
if_<is_alive> :: then_<  
  if_<has3_or_2> :: then_<alive> :: else_<dead>  
> :: else_<  
  if_<has3> :: then_<alive> :: else_<dead>  
>;
```

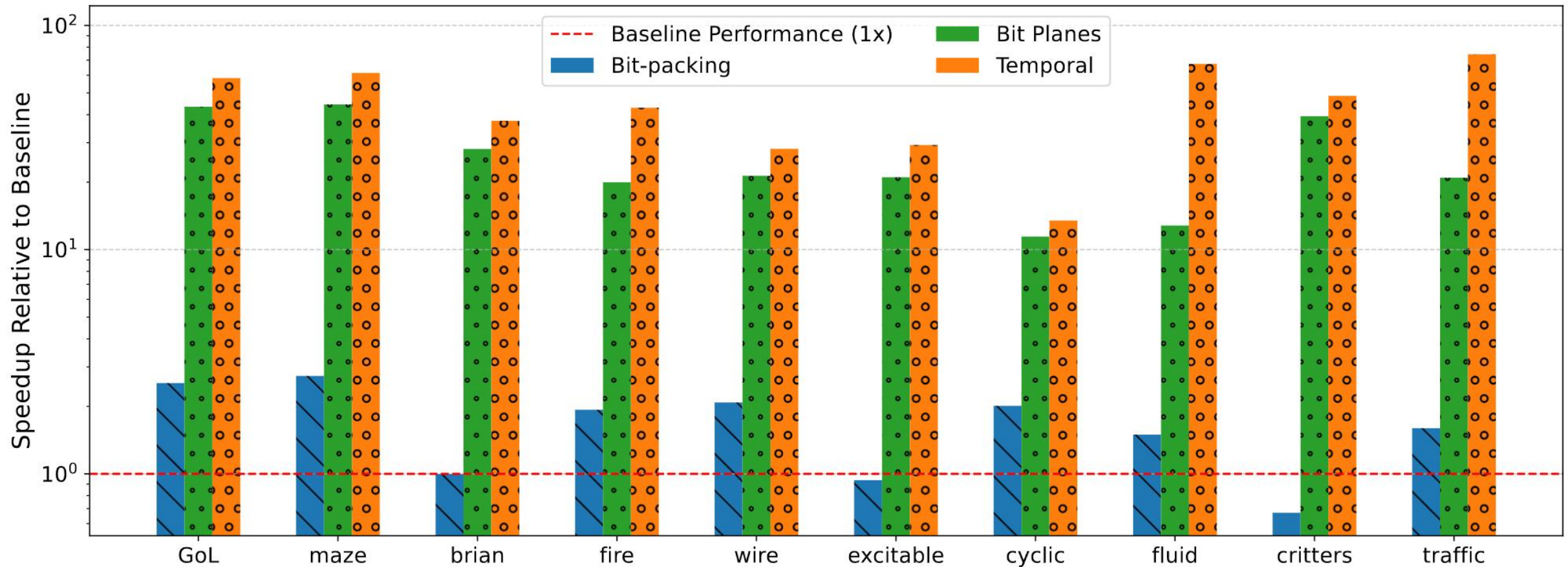
Type-level expression (Cellato DSL)



```
if_<is_alive>::then_<  
  if_<has3_or_2>::then_<alive>::else_<dead>  
>::else_<  
  if_<has3>::then_<alive>::else_<dead>  
>;
```



What the design allows us to gain



Comparison of GPU-parallelized computation of various cellular automata



- Memory access transformations
- Decoupling value encoding and evaluation strategy
- **Design guidance and abstractions' effect on LLM-generated code**

Publications

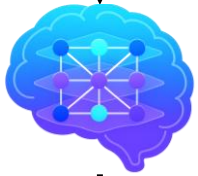
Tutoring LLM into a Better CUDA Optimizer. *Euro-Par, 2025*

Abstractions and Prompting Strategies on LLM-Guided Optimization. *ICA3PP, 2026*

Tutoring LLMs for CUDA

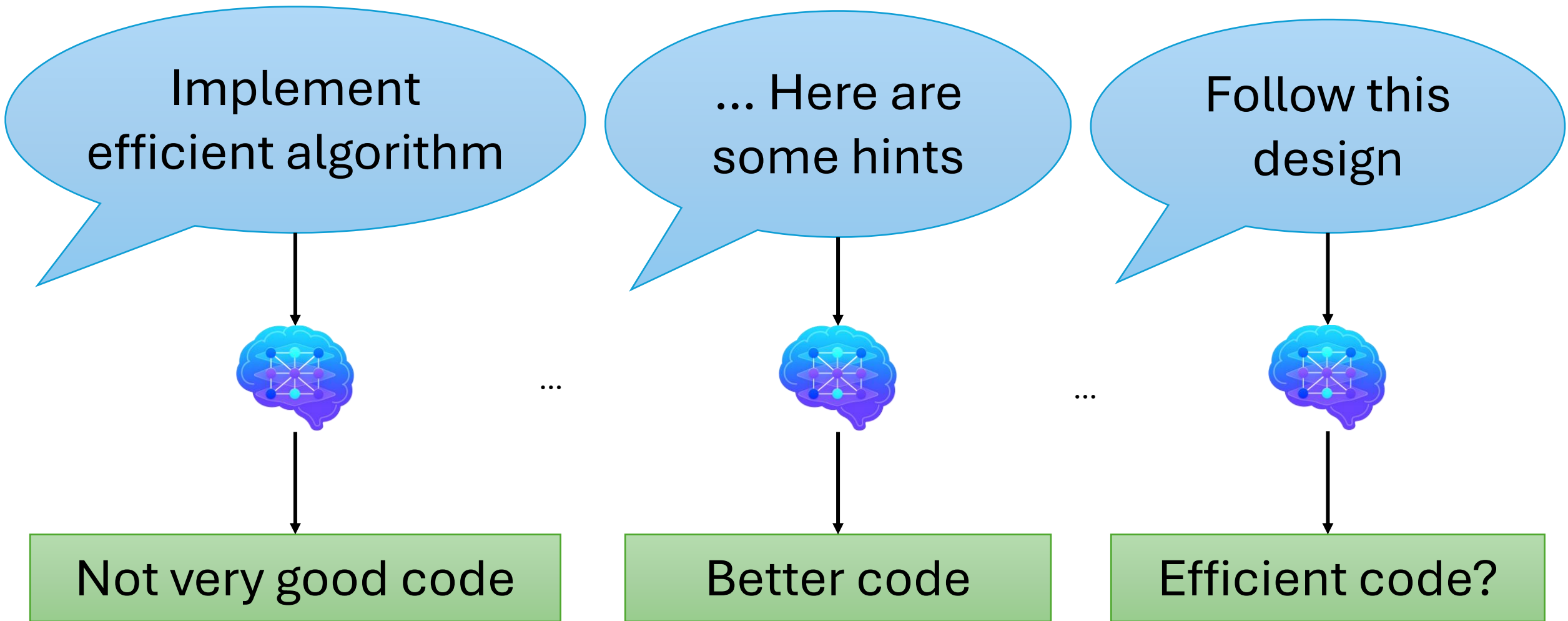


Implement
efficient algorithm



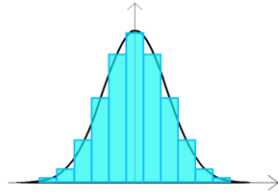
Not very good code

Tutoring LLMs for CUDA

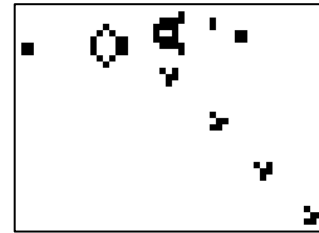


Tutoring LLMs for CUDA

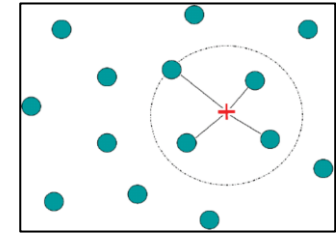
OpenAI o3-mini



Histogram



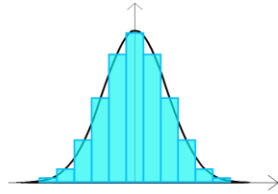
Game of Life



**K-Nearest
Neighbors**

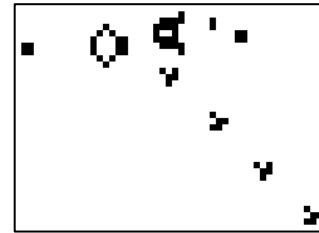
Tutoring LLMs for CUDA

OpenAI o3-mini



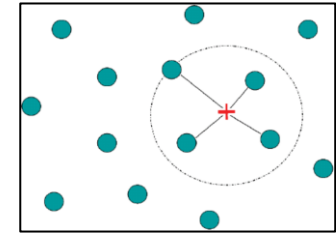
Histogram

Privatization in GPU “shared memory”



Game of Life

Bit-lvl vectorization

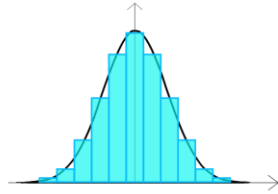


**K-Nearest
Neighbors**

Inter-thread communication

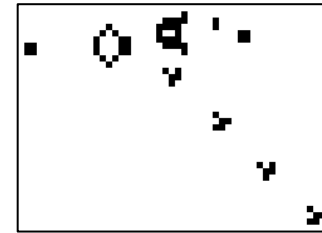
Tutoring LLMs for CUDA

OpenAI o3-mini



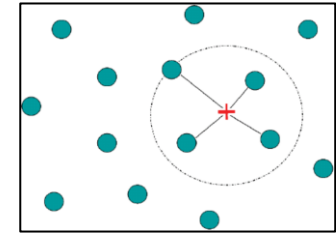
Histogram

Privatization in GPU “shared memory”



Game of Life

Bit-lvl vectorization



**K-Nearest
Neighbors**

Inter-thread communication

Just implement



With hints

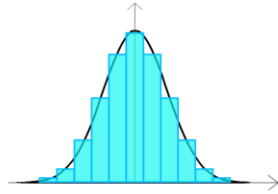


Design direction



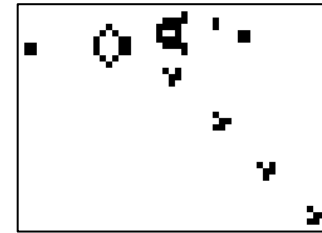
Tutoring LLMs for CUDA

OpenAI o3-mini



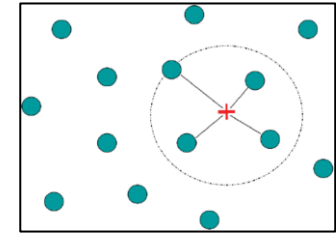
Histogram

Privatization in GPU “shared memory”



Game of Life

Bit-lvl vectorization



**K-Nearest
Neighbors**

Inter-thread communication

Just implement



With hints



Design direction

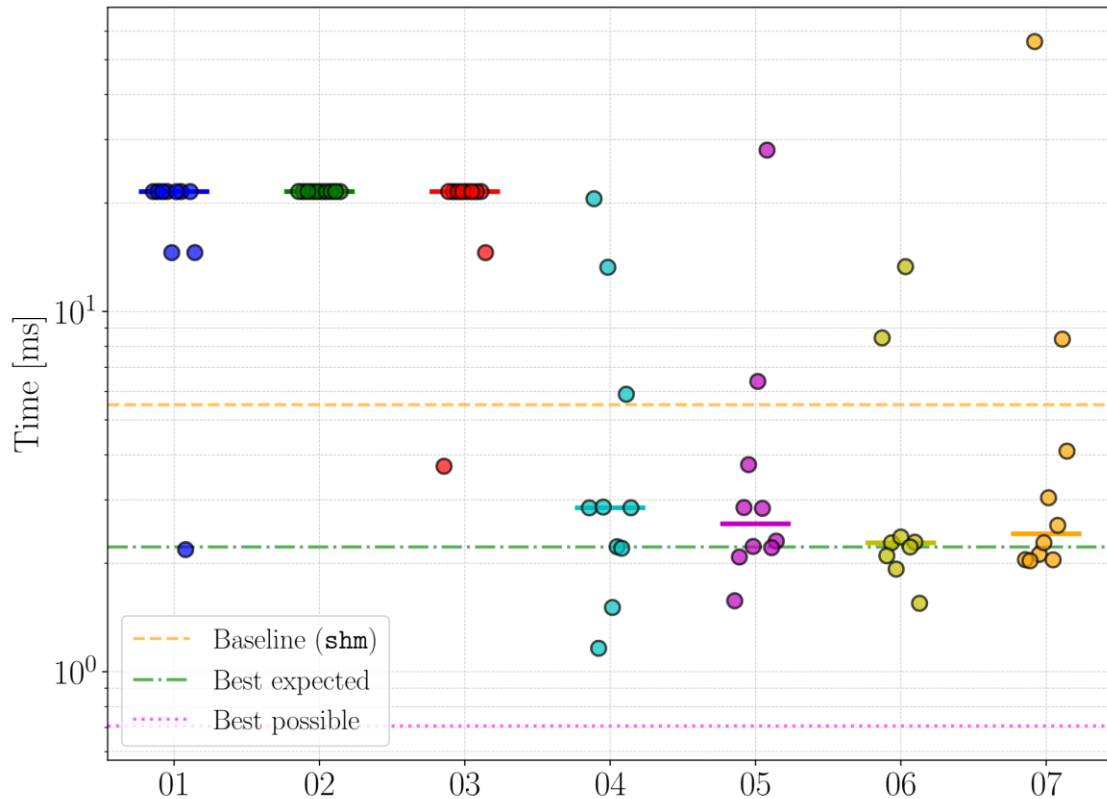


Tutoring LLMs for CUDA – submission

OpenAI o3-mini

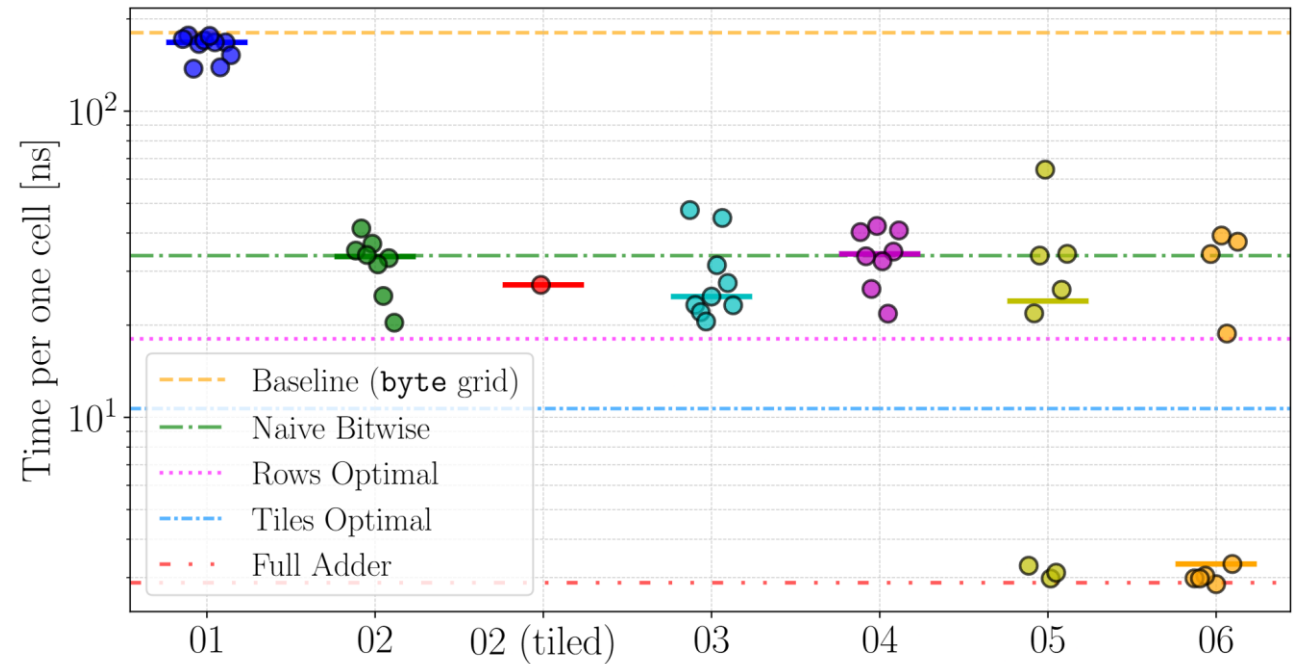


Histogram



Prompt specificity (naïve → ... → design specification)

Game of Life



Prompt specificity (naïve → ... → design specification)

Tutoring LLMs for CUDA – at conference

OpenAI GPT-5

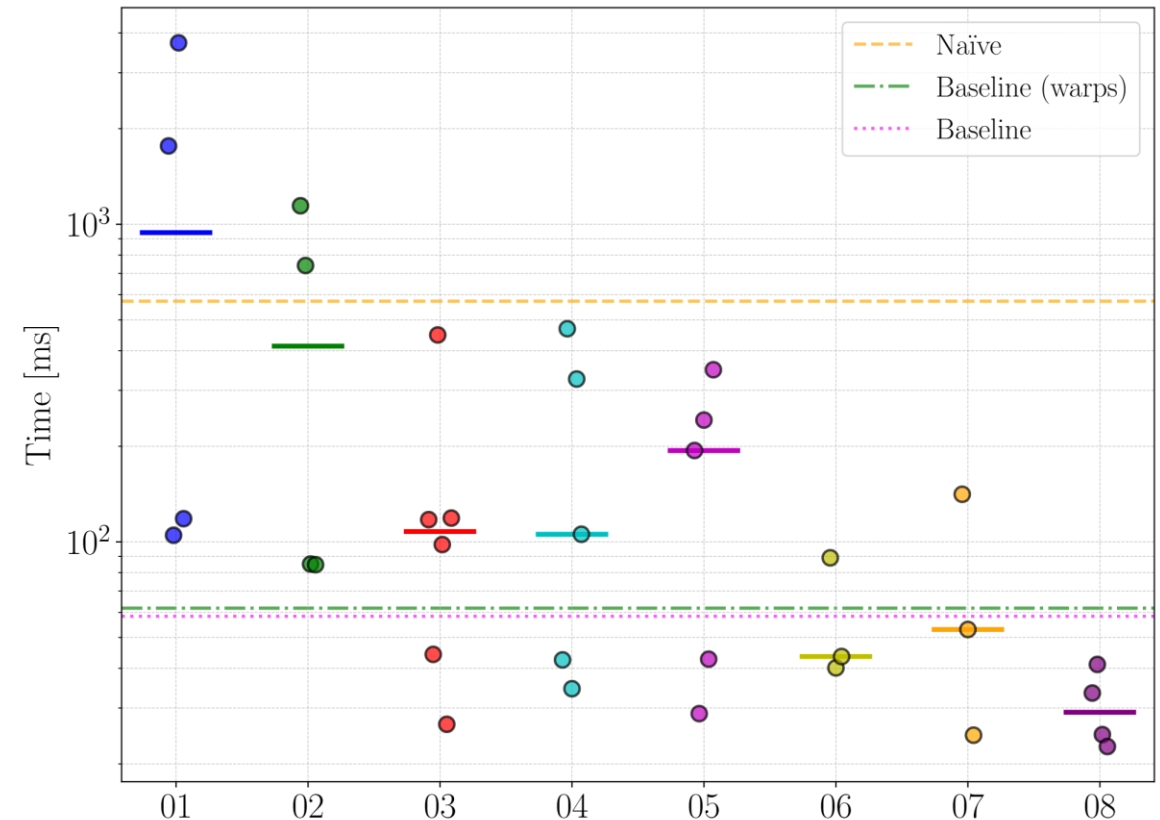


Game of Life



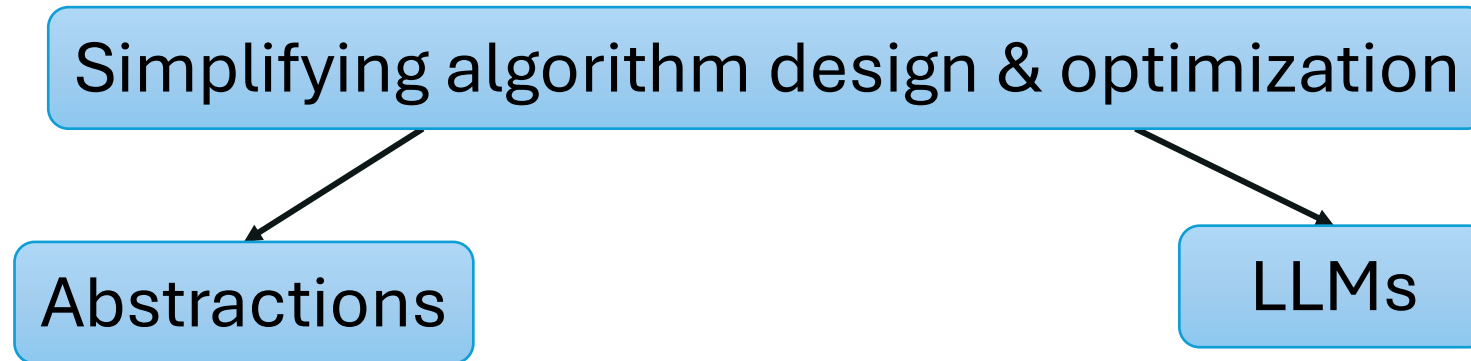
Prompt specificity (naïve → ... → design specification)

k-Nearest Neighbors

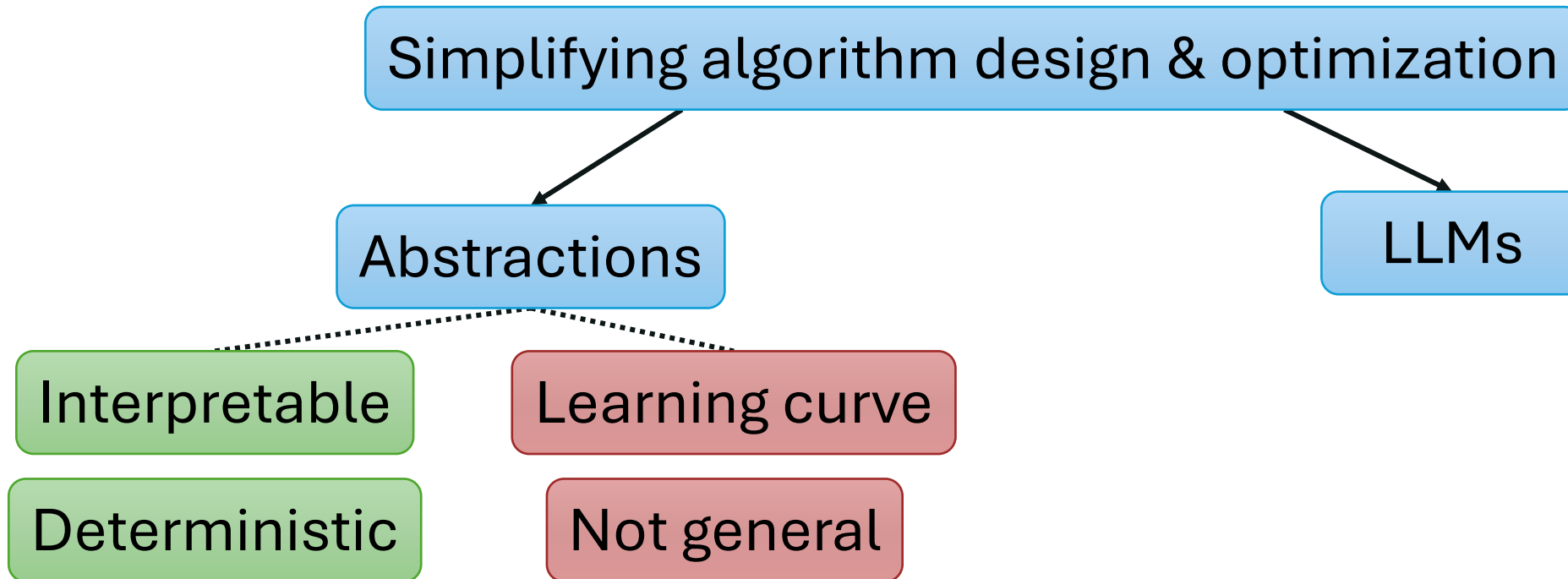


Prompt specificity (naïve → ... → design specification)

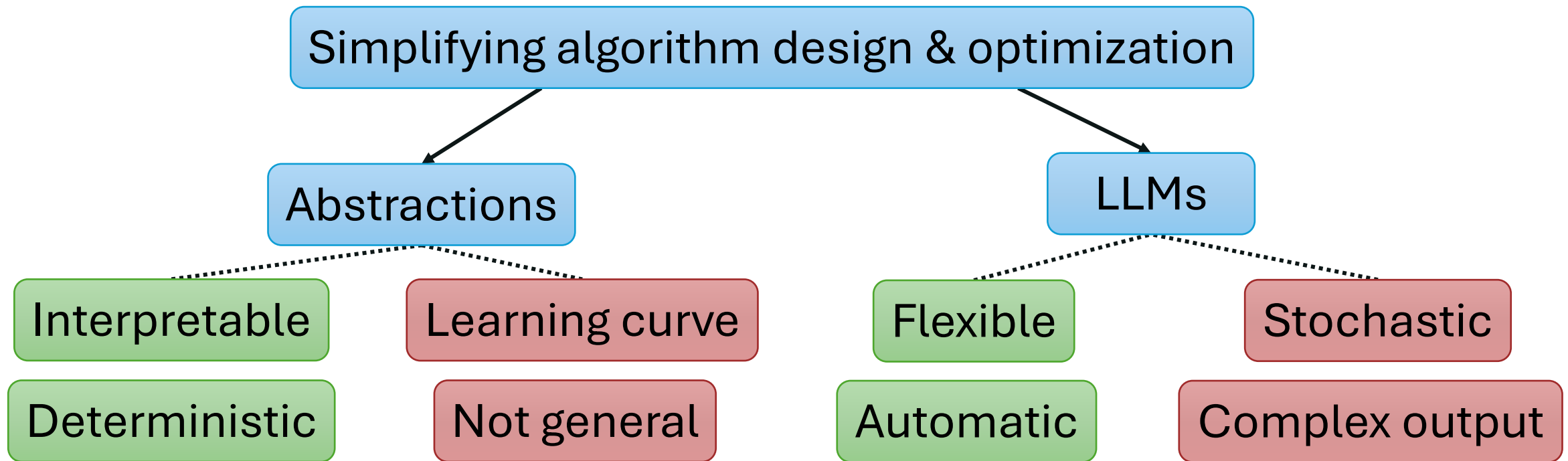
LLMs and abstractions



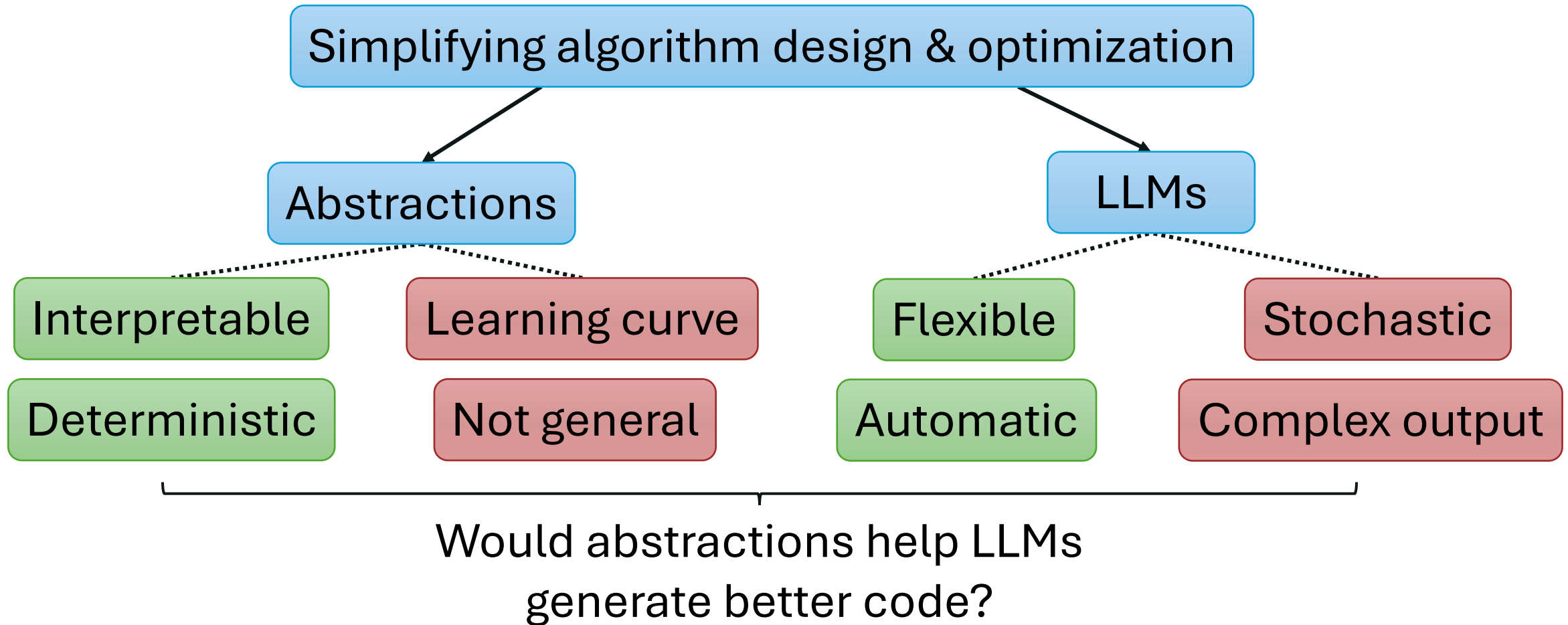
LLMs and abstractions



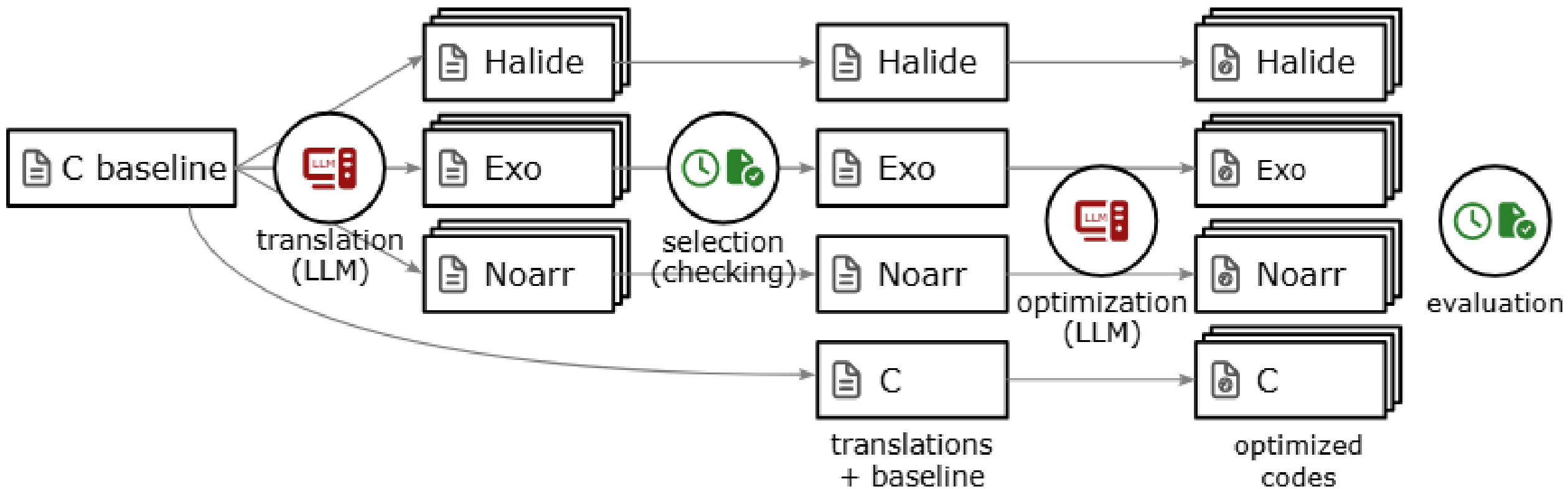
LLMs and abstractions



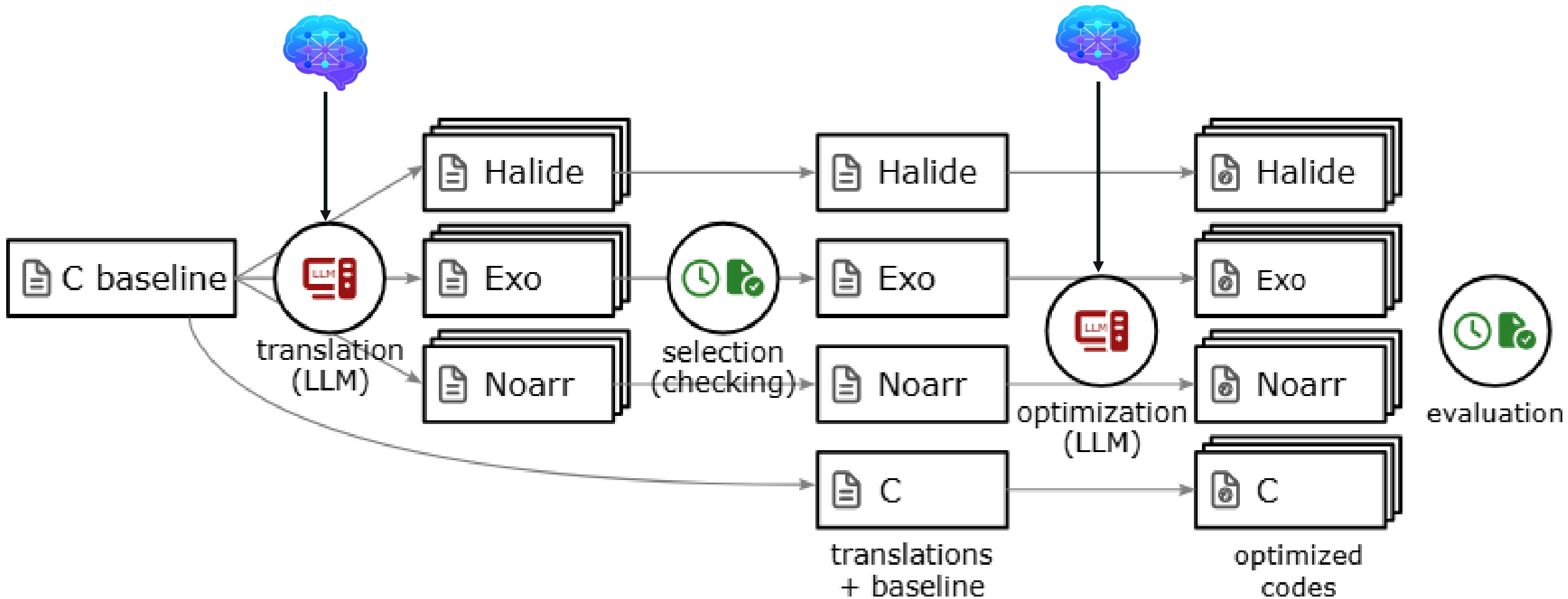
LLMs and abstractions



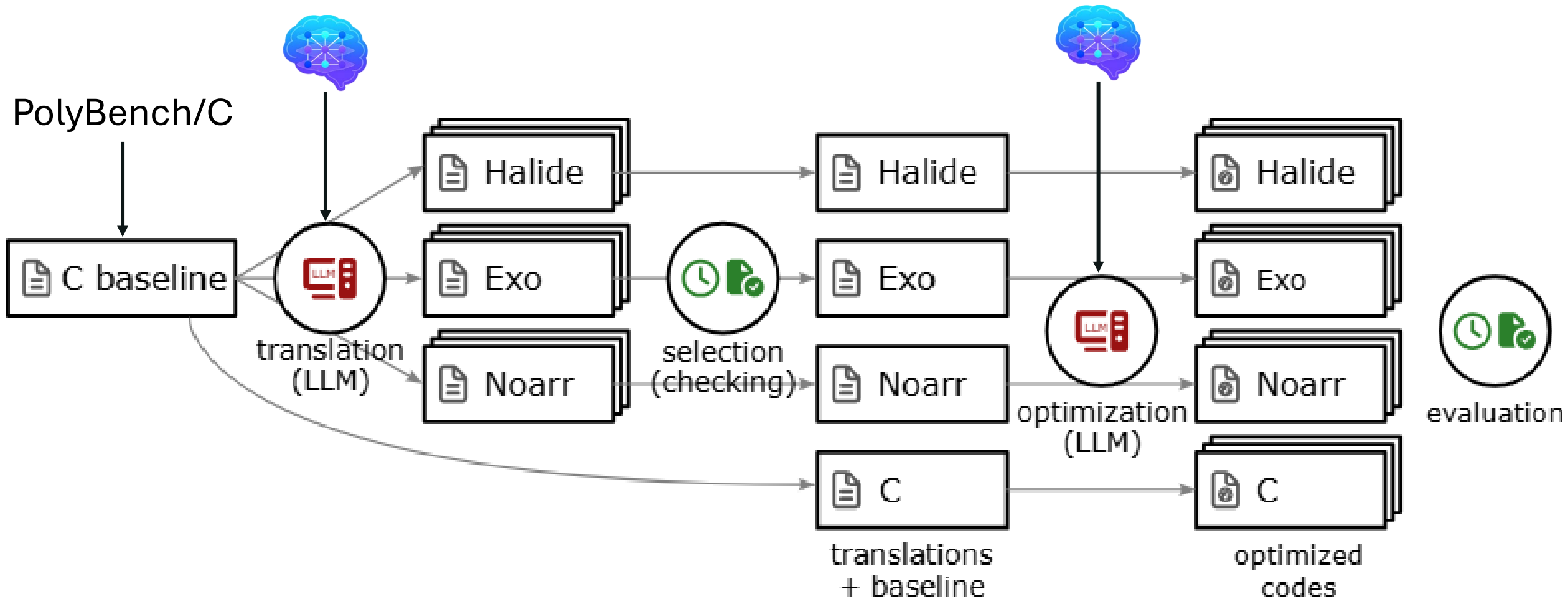
LLMs and abstractions



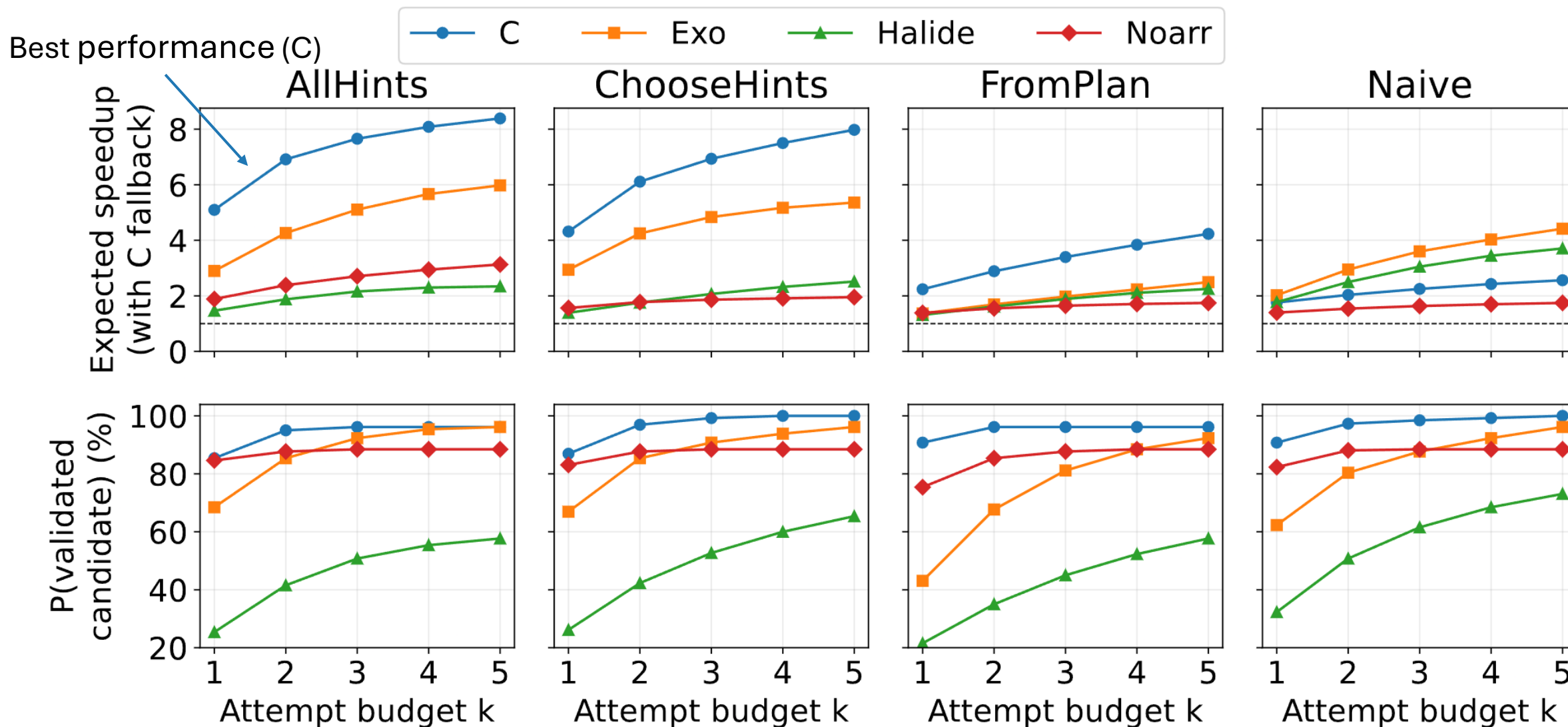
LLMs and abstractions



LLMs and abstractions



C outperforms the abstractions



Answers to research questions



1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?

- Noarr simplifies specifying layouts & traversals, and their transformations
- The approach simplifies parallelization and distributed computing

Answers to research questions



- 1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?**
 - Noarr simplifies specifying layouts & traversals, and their transformations
 - The approach simplifies parallelization and distributed computing
- 2. Can developers write abstract computations and choose optimizations of value encoding and evaluation in specific domains?**
 - Cellato enables users to write update rules in an abstract way
 - Implementation details are applied automatically via prepared components

Answers to research questions



- 1. Can developers specify memory access transformations via C++ abstractions without incurring runtime overhead?**
 - Noarr simplifies specifying layouts & traversals, and their transformations
 - The approach simplifies parallelization and distributed computing
- 2. Can developers write abstract computations and choose optimizations of value encoding and evaluation in specific domains?**
 - Cellato enables users to write update rules in an abstract way
 - Implementation details are applied automatically via prepared components
- 3. How do design guidance and abstractions affect the performance and validity of LLM-generated code?**
 - Specific design hints (or directions) improve the quality of LLM-generated code
 - Abstractions do not significantly improve the performance or the validity

Review comments and questions

Why is Cellato separate from Noarr?



- Optimization tied to bit-level encoding and bitwise evaluation
- Noarr doesn't inspect or abstract statements/expressions
- Cellato adopts layout & traversal agnosticism and the template metaprogramming implementation; the goals are more specific

What do you think an abstraction designed specifically for an LLM should look like?



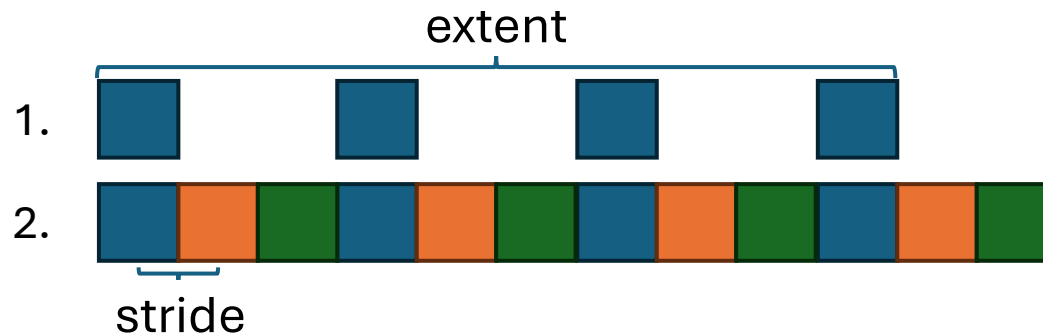
- A good answer would require a specific study
- I have identified (at the time of experiments) that C++ syntax was quite challenging to the LLM (particularly angle brackets)
- Python-based Exo performed best out of the studied abstractions
- Good performance on abstraction-less C suggests more explicit representation might perform better
- Agentic LLMs might benefit from extensive compiler feedback and runtime metrics
- Inspiration by ComPilot: structured back-and-forth communication

Is hvector necessary? vector more convenient?



(context: type_vector uses extent of the underlying type for offset calc.)

- For a general multidimensional structure, hvector is more convenient
 - For the first iteration, vector seems like the right choice
 - For later iterations, the underlying datatype is the previous iteration
 - Then works conveniently only for contiguous-like encodings
 - Not for the following: the extent of sub-layout does not divide the stride of the result layout (see Slide 4 for better visualization of the two communicated layouts)



Is Noarr used outside the department?

If not, why?



- Some little use
 - Adam Šmelko (past member of the department)
 - Marek Čupr (BUT student)
- Noarr is mostly an experimental tool
- Development tied to research
- CuTe?

Can Noarr naturally incorporate `#pragma omp simd` or SIMD-aware traversers



- OpenMP SIMD explicitly rejects class-based iteration (e.g., range-based for)
 - `#pragma omp simd` specifically may be in explicit SIMD-enabled `for_each`
`traverser.omp_simd_for_each(...)`
- More extensive OpenMP pragma support challenging
- Future SIMD-aware traversers
 - Possible with computation-aware abstractions
 - Noarr already implements “planners” ← traversers that store the computation and enable more expressive transformations

Novelty in layout- and traversal-agnosticism



- We do not claim to invent these two concepts as new paradigms
 - The thesis explicitly studies the two concepts
 - introducing them as two identified underexplored principles
- Main novelty
 - Unified approach for layout & traversal-agnostic algorithm design
 - Named dimensions and compositional approach
 - Abstracting evaluation strategy and encoding for Cellular Automata

Which of these ideas can be found in software like Kokkos or OpenMP's loop transformations?



(reaction to layout- and traversal-agnostic design)

- Kokkos Views (and mdspans) abstract physical memory layout
 - Limited to multidimensional arrays
 - Mostly focus row-major vs column-major (and custom strides)
 - Subviews
 - CuTe supports more general layout algebra
- OpenMP (6.0) ← some powerful loop transformation directives
 - Not exactly traversal-agnostic design; good alternative in many use cases

State of the art summary and our novelty



- Layout abstractions
 - Kokkos Views for general use, GridTools for stencils
 - Both target CPU & GPU
 - Both implement facilities for automatic parallelization, tiling (, halo exchange)
- Alternative layout and traversal transformation, parallelization
 - Algorithm + schedule model: Halide, Tiramisu, Exo, OpenMP, ...
 - Tools/frameworks that rely on (custom) compilation pipelines
 - Policy-based execution model: Kokkos, standard C++ algorithms
- Novelty
 - Native C++ compositional approach with named dimensions (Noarr)
 - Sub-byte encoding and bitwise evaluation strategies (Cellato)

Comparison table



	Noarr	Cellato	Kokkos	CuTe	GridTools
Layout-agnostic	✓	✓	✓	✓	✓
Traversal-agn.	✓	✓	Tiling	GPU mapping	✓
Layout composition	✓	✗	✗	✓	✓
Traversal comp.	✓	✗	✗	✓	✓
Parallelization	✓	✓	✓	✓	✓
Distributed ex	✓	✗	✓	✗	✓
Computation	✗	✓	✗	✗	✗
Encoding control	✗	✓	✗	✗	✗
Domain	General	CA	General	General	Stencil
Target	CPU/GPU	CPU/GPU	CPU/GPU	GPU	CPU/GPU
Introduction	2022	2026	2013	2023	2014

Genuine limitations of Noarr and Cellato



- Not paired with formal verification
- Compilation time overhead
- Template metaprogramming imposes some limitations
 - The syntax of the resulting DSLs may be awkward
 - Makes online tuning more costly compared to alternatives such as mdspans or Kokkos Views (recompilation)
- Usability not supported by user studies

Template-exclusive statements



- I do not claim that templates are the only solution
- We do not consider compilation pipeline extensions (e.g., compiler extensions) due to:
 - Such solutions hindering portability; reliance on external tools
 - Core research group focus and expertise
 - Research scope closely tied to algorithm design

Why did the abstractions hinder the LLM



- Some generated Noarr layouts preserve syntactic order of dimensions rather than the semantic one (unintended layout transformations)
- Halide was an awkward representation for many algorithms
- Hallucinated API
 - (mostly for Halide), the LLM considered more general function overloads
 - (mostly for Exo), the LLM failed to preserve expected invariants
 - (mostly for Noarr), the LLM made genuine mistakes (e.g., replaced angle brackets with square brackets)

Contribution summary



	Venue	Year	Analysis	Implementation	Evaluation	Writing
Noarr {	Layouts	ICA3PP	◐	●	◐	◐
	Traversals	PMAM@PPoPP	◐	◐	●	●
	Autotuning	ParCo	●	●	●	●
	Distributed	EuroMPI	●	●	●	●
Cellato {	CA abstraction	JOT	◐	◐	◐	◐
Tutoring LLMs for CUDA		Euro-Par	◐	◐	◐	◐
LLMs and abstractions		ICA3PP	●	●	●	●
Publications listed below are not included in this thesis						
GPU EmbedSOM		GPGPU@PPoPP	◐	◐	◐	◐
Bitwise encoding of GoL		AsHES@IPDPS	◐	◐	◐	◐
kNN with bitonic sort		<i>under review</i> ²	◐	◐	◐	●
Bit-Planes – CA abstr.		<i>under review</i> ³	◐	◐	◐	◐

Contribution map

General HPC computing

Stencil computation



Contribution map

General HPC computing



Layouts

Stencil computation



LLM code generation

Contribution map

General HPC computing



Layouts



Traversals

Stencil computation



LLM code generation

Contribution map

General HPC computing



Layouts



Traversals



Autotuning

Stencil computation



LLM code generation

Contribution map

General HPC computing



Layouts



Traversals



Autotuning

kNN with bitonic sort



k-Nearest Neighbors

Stencil computation



LLM code generation

Contribution map

General HPC computing



Layouts



Traversals



Autotuning

kNN with bitonic sort



k-Nearest Neighbors

Stencil computation



Bitwise encoding of GoL



LLM code generation

Contribution map



General HPC computing



Layouts



Traversals



Autotuning

Stencil computation

Bitwise encoding of GoL



Tutoring LLMs for CUDA



kNN with bitonic sort



LLM code generation

k-Nearest Neighbors

Contribution map



General HPC computing



Layouts



Traversals



Autotuning



Distributed

kNN with bitonic sort



k-Nearest Neighbors

Stencil computation

Bitwise encoding of GoL



Tutoring LLMs for CUDA



LLM code generation

Contribution map



General HPC computing



Layouts



Traversals



Autotuning



Distributed

Knowledge transfer

CA abstraction



Stencil computation

Bitwise encoding of GoL



Tutoring LLMs for CUDA



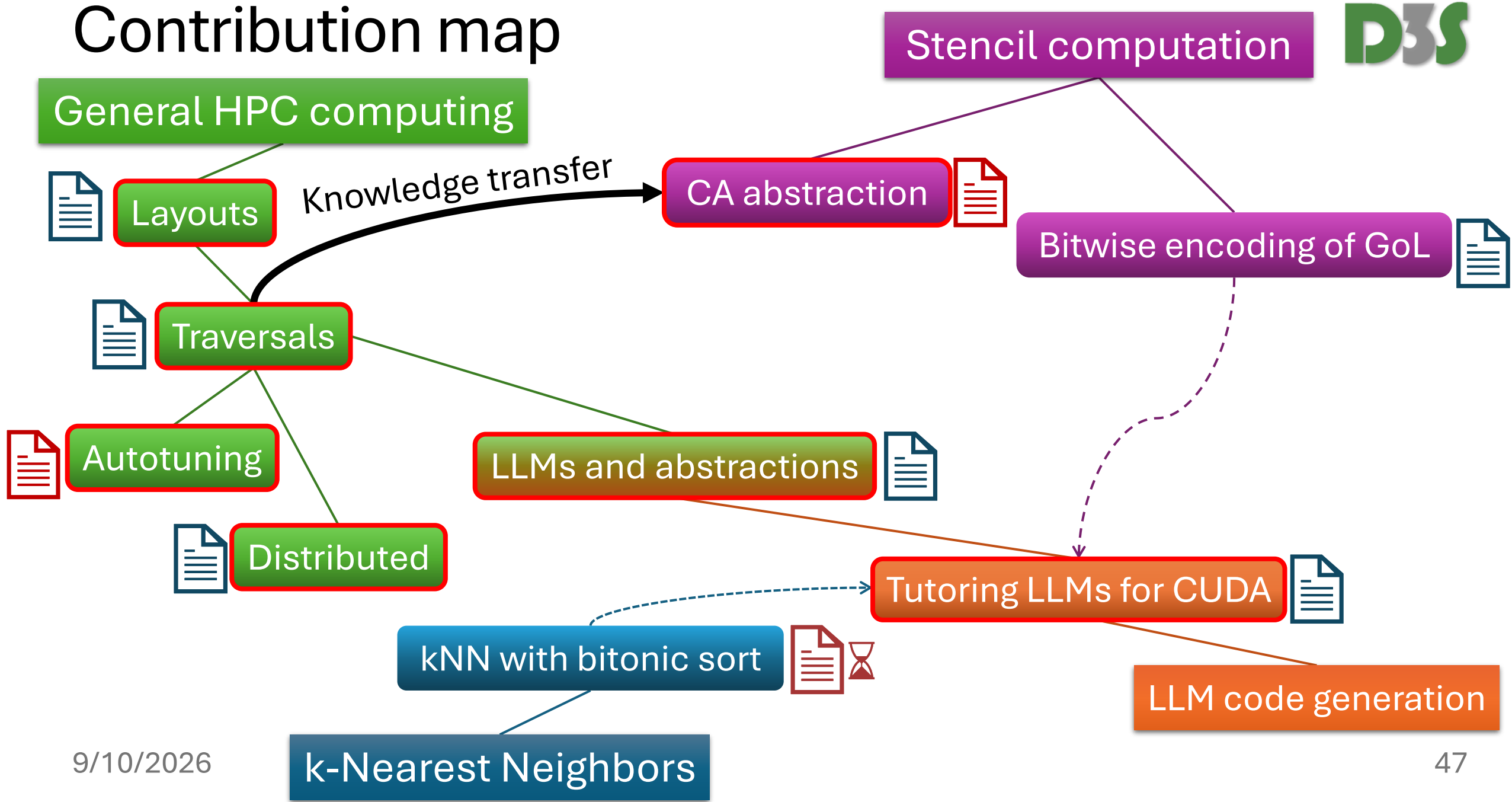
kNN with bitonic sort



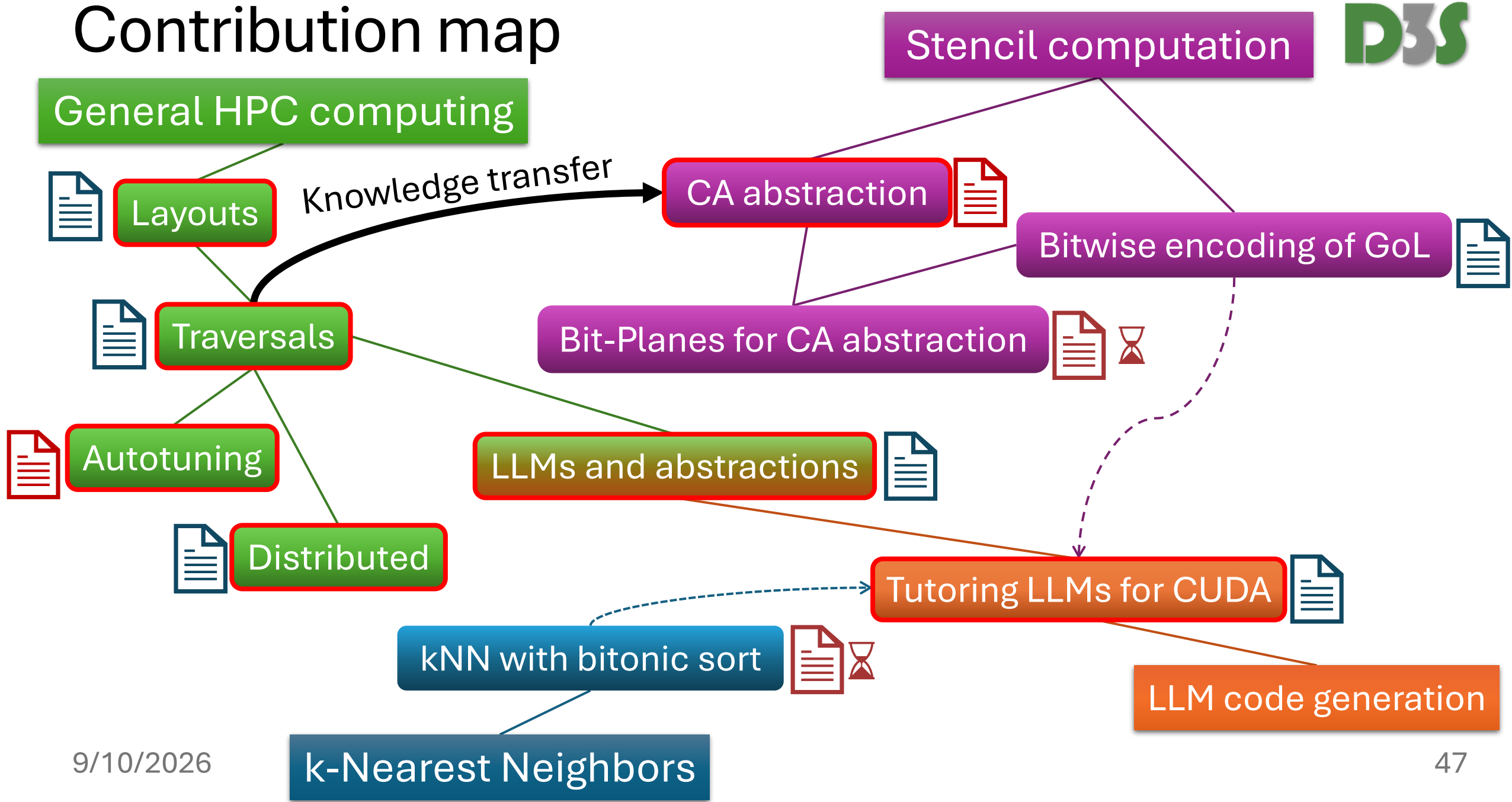
LLM code generation

k-Nearest Neighbors

Contribution map



Contribution map



Autotuning



Autotuning



Choice of layout

Tile size for tiling

Parameters in
the **application**

Autotuning



Choice of layout

Parameters in
the **application**

Tile size for tiling

Parameters in an
autotuner script

Categorical value

Numerical value

Autotuning



Choice of layout

Tile size for tiling

Parameters in
the **application**

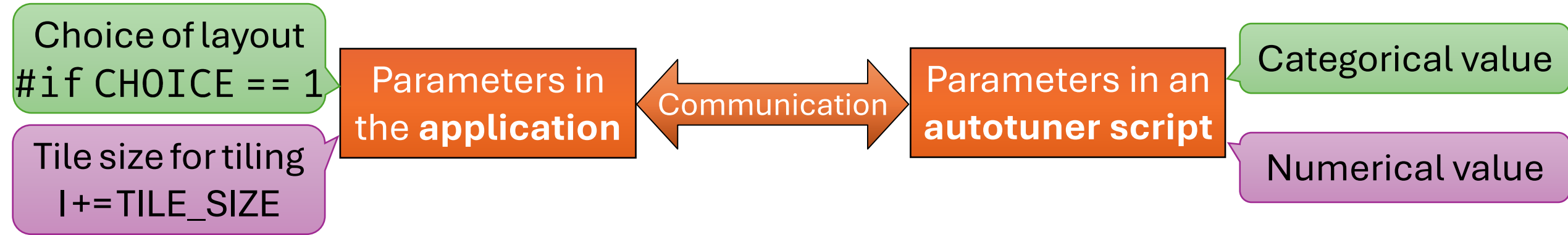
Communication

Parameters in an
autotuner script

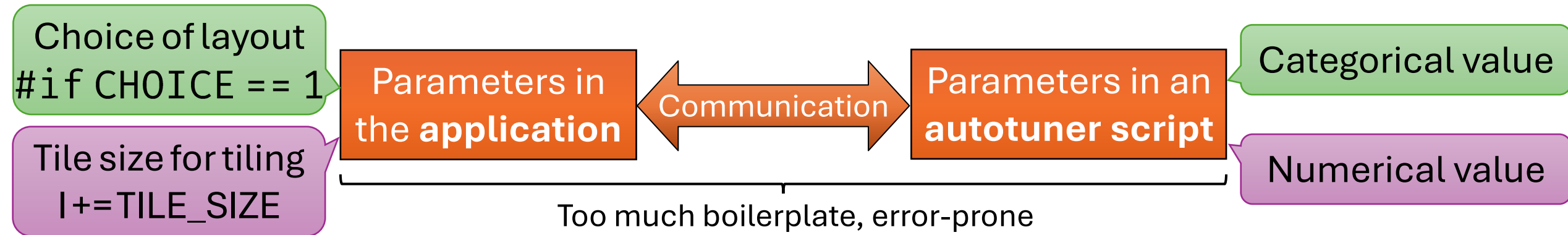
Categorical value

Numerical value

Autotuning



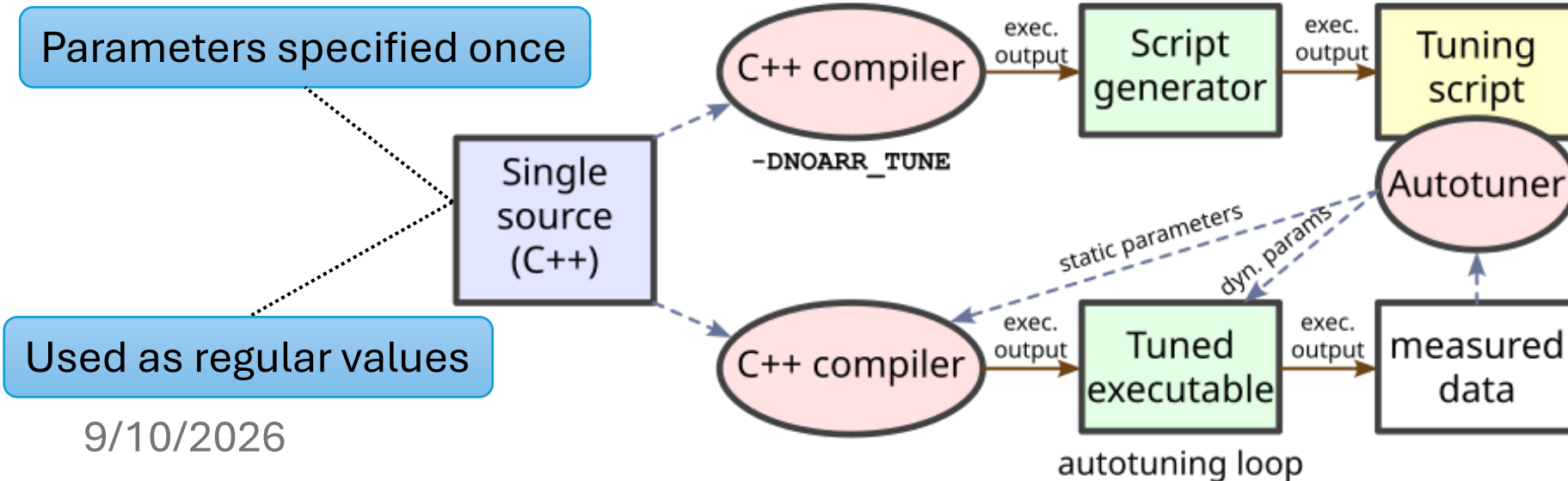
Autotuning



Autotuning



Too much boilerplate, error-prone



Autotuning



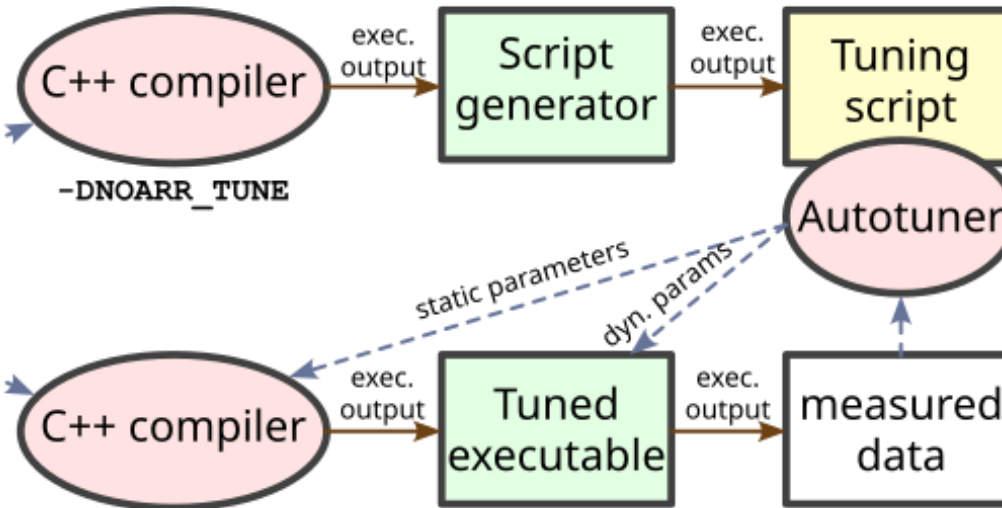
Too much boilerplate, error-prone



Parameters specified once

Single source (C++)

Used as regular values



autotuning loop

ATF

OPTUNA

openuner

Code metrics



Metric	Baseline C	Noarr	Difference
Lines of code	452	393	−13.1%
Characters count	14424	17558	+21.7%
Code tokens	6157	6551	+6.4%
McCabe complexity	189	153	−19.0%
Halstead Difficulty	308.6	371.6	+20.2%
Halstead Effort	$1.38 \cdot 10^7$	$1.86 \cdot 10^7$	+34.9%
Indexation complexity	525	332	−36.8%
Subscript complexity	1126	332	−70.5%

Code metrics



Slightly more verbose

Metric	Baseline C	Noarr	Difference
Lines of code	452	393	-13.1%
<u>Characters count</u>	14424	17558	+21.7%
<u>Code tokens</u>	6157	6551	+6.4%
McCabe complexity	189	153	-19.0%
<u>Halstead Difficulty</u>	308.6	371.6	+20.2%
<u>Halstead Effort</u>	$1.38 \cdot 10^7$	$1.86 \cdot 10^7$	+34.9%
Indexation complexity	525	332	-36.8%
Subscript complexity	1126	332	-70.5%

Code metrics



Slightly more verbose

Metric	Baseline C	Noarr	Difference
Lines of code	452	393	-13.1%
<u>Characters count</u>	14424	17558	+21.7%
<u>Code tokens</u>	6157	6551	+6.4%
<u>McCabe complexity</u>	189	153	-19.0%
<u>Halstead Difficulty</u>	308.6	371.6	+20.2%
<u>Halstead Effort</u>	$1.38 \cdot 10^7$	$1.86 \cdot 10^7$	+34.9%
Indexation complexity	525	332	-36.8%
Subscript complexity	1126	332	-70.5%

Less complex control flow

Code metrics



Slightly more verbose

Metric	Baseline C	Noarr	Difference
Lines of code	452	393	-13.1%
<u>Characters count</u>	14424	17558	+21.7%
<u>Code tokens</u>	6157	6551	+6.4%
<u>McCabe complexity</u>	189	153	-19.0%
<u>Halstead Difficulty</u>	308.6	371.6	+20.2%
<u>Halstead Effort</u>	$1.38 \cdot 10^7$	$1.86 \cdot 10^7$	+34.9%
<u>Indexation complexity</u>	525	332	-36.8%
<u>Subscript complexity</u>	1126	332	-70.5%

Less complex control flow

Easier indexation

Code metrics



Slightly more verbose

Metric	Baseline C	Noarr	Difference
Lines of code	452	393	-13.1%
<u>Characters count</u>	14424	17558	+21.7%
<u>Code tokens</u>	6157	6551	+6.4%
<u>McCabe complexity</u>	189	153	-19.0%
<u>Halstead Difficulty</u>	308.6	371.6	+20.2%
<u>Halstead Effort</u>	$1.38 \cdot 10^7$	$1.86 \cdot 10^7$	+34.9%
<u>Indexation complexity</u>	525	332	-36.8%
<u>Subscript complexity</u>	1126	332	-70.5%

Less complex control flow

Easier indexation

+ Flexibility

+ Type safety

LLM-guided optimization summary



LLM-guided optimization summary



- Prompting specificity 



LLM-guided optimization summary



- Prompting specificity 
- Contemporary abstractions 



LLM-guided optimization summary



- Prompting specificity 
- Contemporary abstractions 
- Possible future research



LLM-guided optimization summary



- Prompting specificity 
- Contemporary abstractions 
- Possible future research
 - Re-evaluate on agentic LLMs
 - Gather feedback (performance, compile errors, ...)
 - Collect necessary context (e.g., work with an abstraction reference)



LLM-guided optimization summary



- Prompting specificity 
- Contemporary abstractions 
- Possible future research
 - Re-evaluate on agentic LLMs
 - Gather feedback (performance, compile errors, ...)
 - Collect necessary context (e.g., work with an abstraction reference)
 - More LLM-focused abstractions and scaffolding



LLM-guided optimization summary



- Prompting specificity 
- Contemporary abstractions 
- Possible future research
 - Re-evaluate on agentic LLMs
 - Gather feedback (performance, compile errors, ...)
 - Collect necessary context (e.g., work with an abstraction reference)
 - More LLM-focused abstractions and scaffolding
- LLMs still do not replace abstractions ( flexibility,  determinism interpretability)